
# SPIN

## Automata-Based Model Checker

**Lorenzo Romandini** 10939723
**Simone Grandi** 10741772
**Edoardo Sartore** 10825131

Prof. **Pierluigi San Pietro**
088882 - **Formal Methods for Concurrent and Real-Time Systems**
A.A. 2024-2025

# Overview

- Model checking
- History
- Introduction
- Key features
- Setup
- Command options
- Theoretical background
- Promela
- Extensions
- Real-World Applications
- Bibliography
- Demo


POLITECNICO
MILANO 1863

# Model checking

# What is Model checking?

Recap

"*Model checking is an automated technique that, given a finite-state* ***model*** *of a system and a logical* ***property****, systematically checks whether this property holds for (a given initial state in) that model.*"

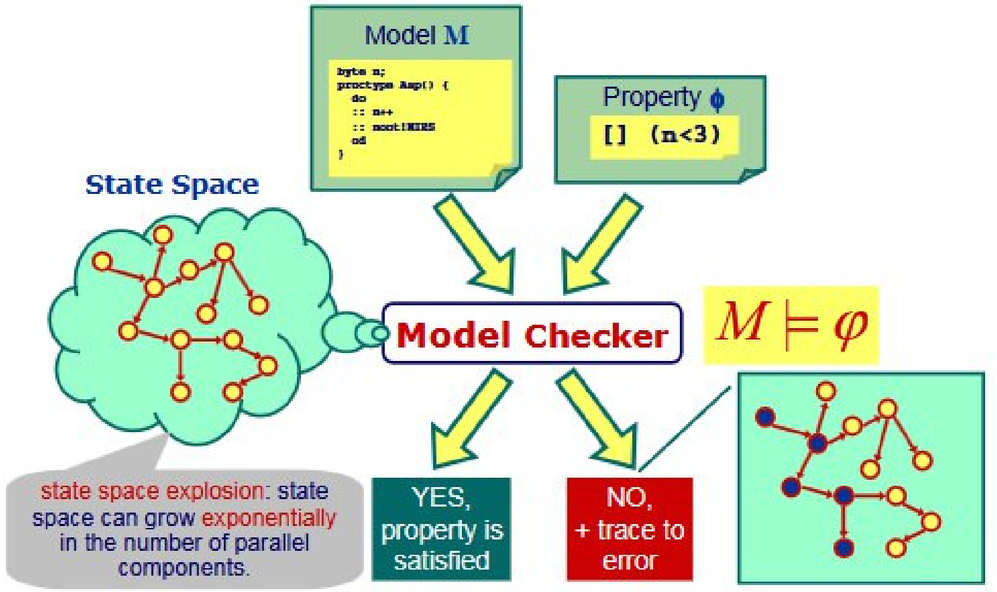
Clarke & Emerson, 1981

<u>Advantages</u>:

- automatic
- if property doesn't hold, a counterxample is produced

<u>Disadvantages</u>:

- state space explosion



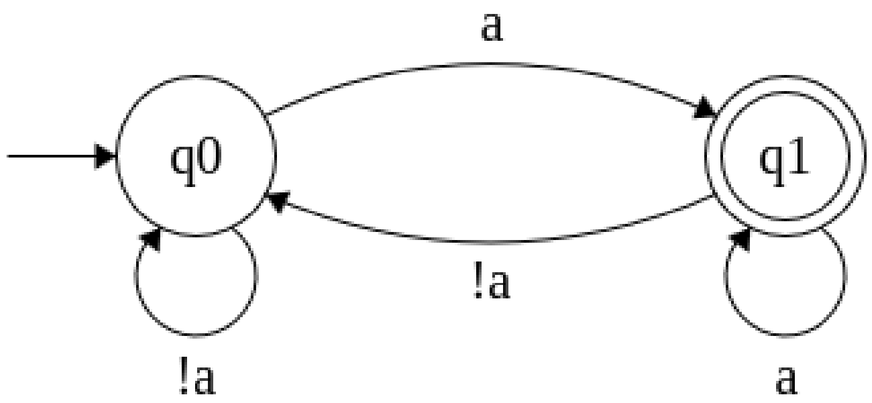
# ω-properties

Recap

An **ω-property** is a set of *infinite* words over an alphabet.

An **ω-automata** (like Büchi Automata) can be used to represent ω-properties.

example: "*infinitely often a*"



# Linear-time properties

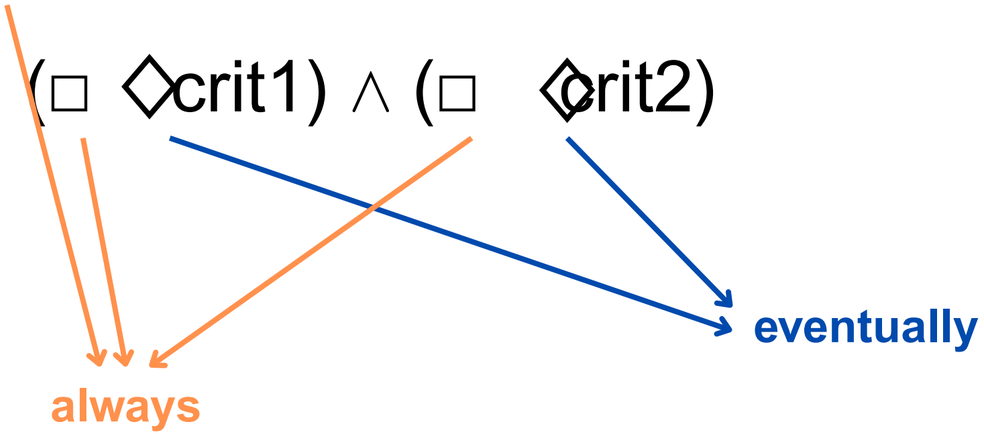
Recap

A **linear-time** (LT) property is a set of *infinite* traces (sequence of state labels) that represents the desired behaviours of the system.

**Linear Temporal Logic** (LTL) formulae can be used to express LT properties to be checked for a system.

*examples*: mutual exclusion: $\Box(\neg crit1 \vee \neg crit2)$

starvation freedom: $(\Box \Diamond crit1) \wedge (\Box \Diamond crit2)$



# Examples of properties

Recap

- **Safety:** any infinite word that violates the property contains a bad prefix, so the violation is detected in finite traces.
  informally*: "nothing bad happens"*
  *e.g.*: only one process at a time can be inside the critical section (mutual exclusion)
- **Liveness:** any (valid or not valid) infinite word has a finite prefix that is not ruled out, so the violation is detected in infinite traces.
  informally*: "something good eventually happens"*
  *e.g.*: each waiting process will eventually enter its critical section (starvation freedom)
- **Fairness:** assumptions that restrict the set of execution traces of the system; they are required to prove certain liveness properties.
  informally*: "unrealistic scenarios are ruled out"*
  *e.g.*: if a process is continuously requesting, it will eventually be granted access

POLITECNICO
MILANO 1863

# History

# History

- 1980 **-** ***Pan*****:** one-pass, on-the-fly verification system for safety properties.
- 1983 **-** ***Trace*****:** shifted to automata-based verification for efficiency.
- 1989 **-** ***SPIN*****:** teaching tool, checks omega-regular properties, including LTL.
- 1994 - **Partial Order**: included static partial order reduction to save memory.
- 1995 - **LTL conversion**: added algorithm for converting from LTL syntax to automata.
- 1997 **- Minimized Automata:** optimized state representation.
- 2003 **-** v***4.x*****:** embedded C code; breadth-first search (BFS); data abstraction.
- 2007-2008 **-** v***5.x*****:** model checking on multicore systems, swarm verification.
- 2010-2012-2016 **-** v***6.x*****:** multiple never claims, inline LTL, parallel BFS, open source.

Originally a teaching tool for protocol verification, SPIN has evolved into a powerful, open-source model checker for concurrent systems, supporting advanced verification techniques.

POLITECNICO
MILANO 1863

# Introduction

# SPIN (Simple Promela INterpreter)

- SPIN is a **verification system** distinguished by its focus on *multi-threaded software* and its use of the PROMELA language for system descriptions

- Supports **LTL model checking**, basic *safety* and *liveness* properties (with or without LTL), assertions, Büchi automata, and **ω-regular properties**

- SPIN is an **automata-based** model checker because employs an "on-the-fly" approach, meaning it constructs the necessary automata (like Büchi automata for temporal logic) as it explores the state space of the system

- In particular SPIN uses a series of optimization to avoid the problem where the number of possible states in a system's model grows exponentially with the complexity of the system (**state space explosion**)


SPIN first logo

# SPIN usages

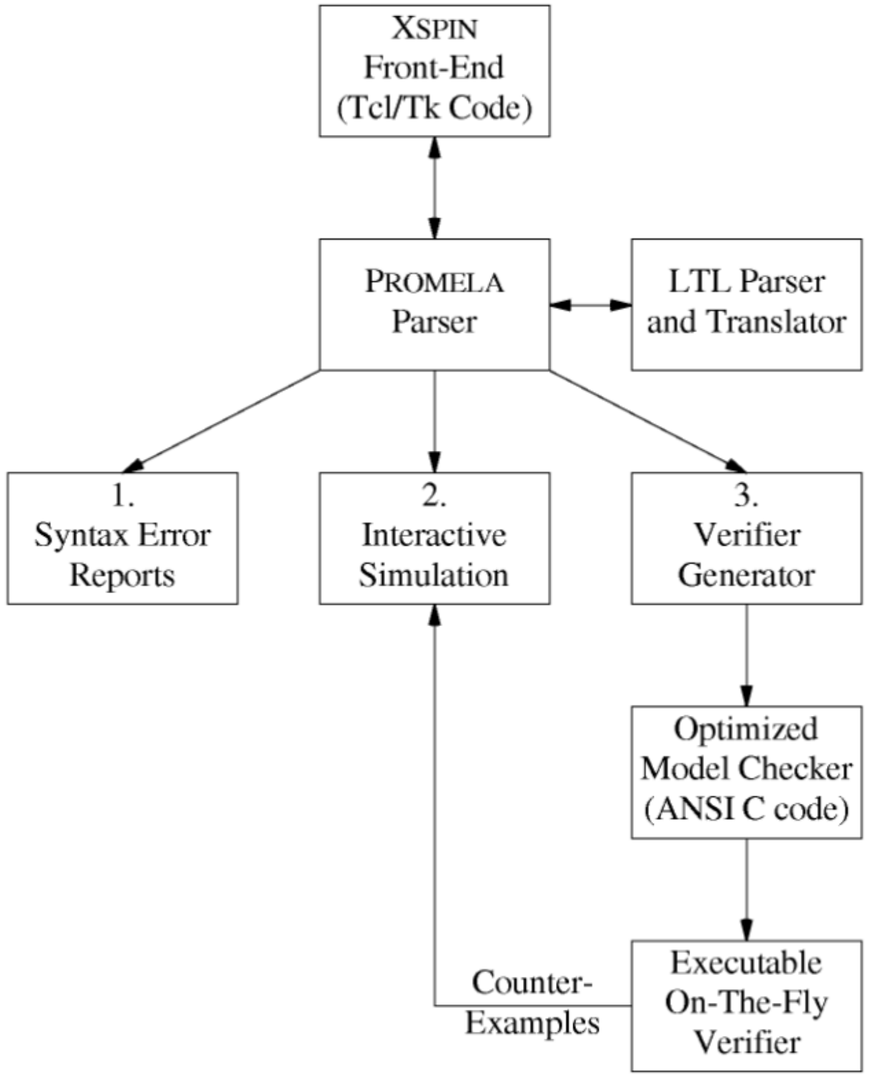
- **Simulator**: rapid prototyping with various simulation types.
- **Exhaustive Verifier**: rigorously proves correctness requirements using partial order reduction.
- **Proof Approximation System**: validates large models with maximal state space coverage.
- **Driver for Swarm Verification**: utilizes cloud networks for parallel and diversified defect searching in very large models.

# Properties check

- **Safety** properties:
  SPIN looks for a trace leading to the bad prefix.
  If there is not such a trace, the property is satisfied.

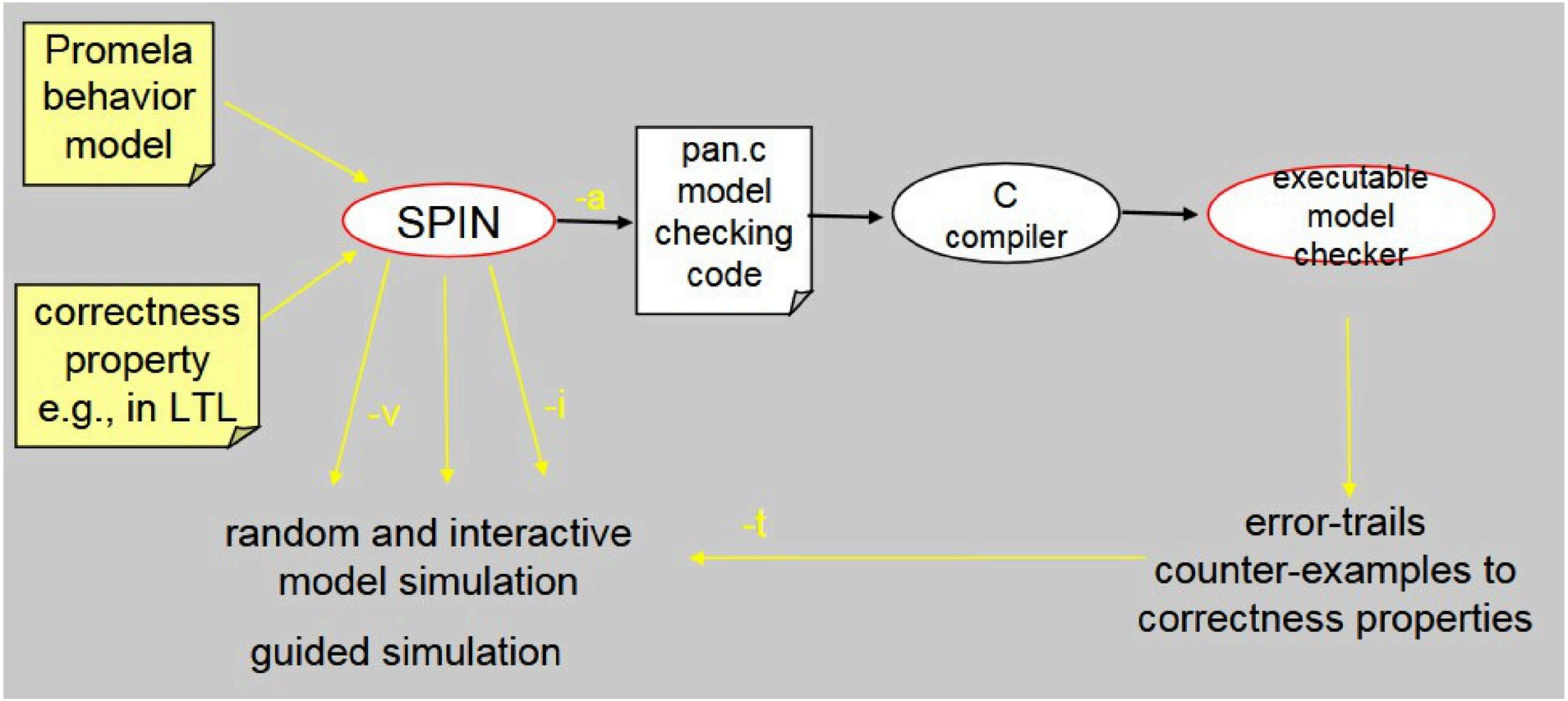
- **Liveness** properties:
  SPIN looks for a (infinite) loop in which the “good” thing does not happen. If there is not such a loop, the property is satisfied.

# Structure




- **Model Specification:**
  - Create a high-level model of the concurrent system or distributed algorithm using PROMELA (Process Meta Language). Optionally you can employ XSPIN for a graphical interface.
- **Simulation:**
  - Perform random/interactive/guided simulation to validate the model to gain confidence in the model's basic behavior and identify potential errors early.
- **LTL Specification & Claim Generation:**
  - Specify correctness properties using Linear Temporal Logic (LTL) and then generate PROMELA "never-claims" from LTL formulas.
- **Verification Program Generation:**
  - SPIN generates an optimized verification program (C code). Choose reduction algorithms at compile time (e.g., partial order reduction) to manage state explosion.
- **Verification Execution:**
  - Compile and execute the verification program, then systematically explore the state space and check for violations of the correctness claims.
- **Counterexample Analysis:**
  - If a violation is found, SPIN provides a counterexample (execution trace). Feed the counterexample back into the simulator and analyze the trace to understand the error's cause.

# Workflow


Promela behavior model
correctness property e.g., in LTL
SPIN
-a
pan.c model checking code
C compiler
executable model checker
-v
-i
random and interactive model simulation
guided simulation
-t
error-trails
counter-examples to correctness properties

POLITECNICO
MILANO 1863

# Key features

# Key features

✅ **Modeling**

🔎 **Verification**

🧠 **Simulation**

🐞 **Error Detection & Traces**

# Modeling

Specify system behavior using the **Promela language**

- We can describe the systems dividing it into **processes**
- And we can model their comunication using **channels/global variables**
- It’s optimal to describe **non-deterministic choises** and **concurrent execution**

**Automata generation**: automatically translates Promela models into Finite State Automata to represent system behavior.

# Verification

Check logical correctness of models:

- **Deadlocks**
- **Race conditions**
- **Unreachable code**
- **Invalid end states**
- Check **assertions** embedded in the Promela code
- Check **temporal logic properties** using Linear Temporal Logic (LTL) formulas.
- **Swarm verification**: is a tool that runs many randomized verification strategies in parallel, increasing the chances of catching rare bugs

Spin uses **partial order reduction** techniques to minimize the number of states it needs to explore, making verification of large systems more feasible.

# Simulation

- **Interactive simulation**: Step through executions manually.
- **Random simulation**: Explore different execution paths to discover unexpected behaviors.
- Understand how a concurrent system behaves in different scheduling scenarios.

For *simulation* as for the *verification,* instead of using the CLI we can use iSPIN, the SPIN GUI,for a better user experience

# Error Detection & Traces

- Automatically generates **counterexamples** when a property fails.
- Provides **execution traces** that show the exact sequence of events leading to an error.
- These traces can be visualized in tools like iSpin.
- Can also generate trail files for failed runs.
- Useful to visualize state transitions and message passing.

POLITECNICO
MILANO 1863

# Setup

# Windows

1. Download SPIN suitable binary https://github.com/nimble-code/Spin/tree/master/Bin and extract the compressed executable file
2. Download Cygwin https://www.cygwin.com/install.html and install it (make sure to add the following packages: ***bison***, ***gcc-core***, ***graphviz***, ***make, tcl***, ***tcl-tk*** )
3. Rename the executable downloaded before to "*spin.exe*" and move it inside the "cygwin/bin" folder
4. Add "/cygwin/bin" to the System PATH environment variable

# Linux 

**Prerequisite.** Make sure to have a C compiler (**gcc**) installed in your system

1. Clone GitHub repository https://github.com/nimble-code/Spin/

```
$ git clone https://github.com/nimble-code/Spin.git
```

2. Move into “Spin/Src”directory and compile the files inside

```
$ cd Spin/Src

$ make
```

3. Copy the executable into “/usr/bin” to be able to run **spin** from anywhere

```
$ cp spin /usr/bin
```

POLITECNICO
MILANO 1863

# Command options

# Run Time Options for SPIN

## Simulation options

| | |
|---|---|
| **-B** | Suppresses the verbose printout at the end of a simulation run (giving process states etc.). |
| **-b** | Suppresses the execution of printf statements within the model (see also -T). |
| **-c** | Produce an ASCII approximation of a message sequence chart for a random or guided (when combined with -t) simulation run. See also option -M. |
| **-g** | Shows at each time step the current value of global variables (see also -w). |
| **-i** | Perform an interactive simulation, prompting the user at every execution step that requires a nondeterministic choice to be made. The simulation proceeds without user intervention when execution is deterministic. |
| **-jN** | Skip the first N steps of a random or guided simulation. (See also option -uN.) |
| **-l** | In combination with option -p, shows the current value of local variables of the process (see also -w). |
| **-M** | Produce a message sequence chart in Postscript form for a random simulation or a guided simulation (when combined with -t), for the model in file , and write the result into file.ps . See also option -c. |
| **-nN** | Set the seed for a random simulation to the integer value N. There is no space between the -n and the integer N. -p, shows the current value of local variables of the process. |
| **-p** | Shows at each simulation step which process changed state, and what source statement was executed. |
| **-qN** | In columnated output (option -c) and elsewhere, suppress the printing of output for send or receive operations on the channel numbered N. |
| **-r** | Shows all message-receive events, giving the name and number of the receiving process and the corresponding the source line number. For each message parameter, show the message type and the message channel number and name. |
| **-s** | Shows all message-send events. |
| **-T** | Suppress the default indentation of output from print statements. By default the output from process i is indented by i spaces. See also option -b. |
| **-t[N]** | Perform a guided simulation, following the error trail that was produces by an earlier verification run, see the online manuals for the details on verification. If an optional number is attached (no space between the number and the -t) the error trail with that sequence number is opened, instead of the default trail, without sequence number. |
| **-uN** | Stop a random or guided simulation after the first N steps. (See also option -jN.) |
| **-v** | Verbose mode, adds some more detail, and generates more hints and warnings about the model. |
| **-w** | Even more verbose output with options -l and -g (e.g., prints all variable values, not just those that change). |

# Run Time Options for SPIN

## Generate, Compile and Run

The **-search** flag controls the overall process of generating, compiling, and running the verifier, bridging SPIN and the verifier (**pan**).

Options like *-bfs*, *-bcs*, etc., modify how the verifier operates during execution, affecting the search strategy, property checking, or optimization techniques.

| | |
|---|---|
| **-search** | Generates verifier source code (pan.c), compiles, and runs it; passes subsequent options to compiler (-[ODUE]) or verifier (others). Options for SPIN parsing (e.g., macros) must *precede* **-search**. |
| **-bfs**, **-bfspar**, **-bcs, -ltl name**, **-safety**, **-noclaim**, **-bitstate**, **-hc**, **-collapse**, **-i**, **-l**, **-a** | Options passed *after* **-search**:<br>Enable BFS, parallel BFS, bounded context switching, LTL property, safety checks, disable never claim, bitstate hashing, hash-compact, collapse compression, interactive mode, local variable display, or acceptance cycle detection. |

# Run Time Options for SPIN

## Verification Generation

| | |
|---|---|
| **-a** | Generate a verifier (model checker) for the specification. The output is written into a set of C files, named pan.[ cbhmt ] that can be compiled, (e.g., cc pan.c ) to produce an executable verifier. The online Spin manuals (see below) contain the details on compilation and use of the verifiers. |
| **-A** | Perform property-based slicing, warning the user of all statements and data objects that are likely to be redundant for the stated properties (i.e., in assertions and never claims). |
| **-d** | Produce symbol table information for the model specified in file . For each Promela object this information includes the type, name and number of elements (if declared as an array), the initial value (if a data object) or size (if a message channel), the scope (global or local), and whether the object is declared as a variable or as a parameter. For message channels, the data types of the message fields are listed. For structure variables, the 3rd field defines the name of the structure declaration that contains the variable. |
| **-F** | file This behaves identical to option -f but will read the formula from the file instead of from the command line. The file should contain the formula as the first line. Any text that follows this first line is ignored, so it can be used to store comments or annotation on the formula. (On some systems the quoting conventions of the shell complicate the use of option -f. Option -F is meant to solve those problems.) |
| **-f** | LTL Translate the LTL formula LTL into a never claim.<br>This option reads a formula in LTL syntax from the second argument and translates it into Promela syntax (a never claim, qhich is Promela's equivalent of a Buchi Automaton). The LTL operators are written: [] (always), <> (eventually), and U (strong until). There is no X (next) operator, to secure compatibility with the partial order reduction rules that are applied during the verification process. If the formula contains spaces, it should be quoted to form a single argument to the Spin command. As the name suggests, a Spin never claim is used to specify behavior than is required to be impossible, i.e., behavior that would violate a user-specified property. This means that to check for compliance with an LTL formula, the formula must be negated explicitly before it is converted into a never claim. Negating an LTL formula is easy: just place the formula "f" in parentheses and negate it: "!(f)". |
| **-J** | Reverse the evaluation order of nested 'unless' statements (so that it conforms to the evaluation order of nested 'catch' statements in Java). |
| **-m** | Changes the semantics of send events. Ordinarily, a send action will be (blocked) if the target message buffer is full. With this option a message sent to a full buffer is lost. |
| **-V** | Prints the Spin version number and exits. |

# Run Time Options for SPIN

## Optimization

The following group of options controls which optimizations are enabled or disabled for verifier generation.
By default most of the important optimizations are enabled.

| | |
|---|---|
| **-o1** | Disables dataflow optimizations (marks variables as dead, resets to zero; may increase complexity). |
| **-o2** | Disables dead variable elimination (rarely needed; confirms effect on state vector/ memory). |
| **-o3** | Disables statement merging (makes **pan -d** output more explicit, loses merging benefits). |
| **-o4** | Enables experimental rendez-vous optimizations (precomputes feasibility; no runtime benefit seen). |
| **-o5** | Disables case caching (faster compilation with caching enabled; disable only for bugs). |
| **-o6** | Reverts to pre-6.2 rules for priority tags interpretation. |
| **-o7** | Reverts to pre-6.3 rules for semicolon usage. |

# Run Time Options for PAN

**Pan** is a model-specific verifier generated by Spin from a Promela model (using `$ spin -a file.pml` ).

It compiles C code (pan.c) into an executable (**pan**) that performs exhaustive or partial state-space verification to check for errors like deadlocks, assertion violations, or LTL property violations, complementing Spin's random simulation capabilities.

| | |
|---|---|
| **-A** | suppress the reporting of assertion violations (see also **-E**) |
| **-a** | find acceptance cycles (available if compiled *without* **-DNP**) |
| **-b** | bounded search mode, makes it an error to exceed the search depth, triggering and error trail |
| **-cN** | stop at *N*th error (defaults to first error if *N* is absent) |
| **-d** | print state tables and stop |
| **-E** | suppress the reporting of invalid endstate errors (see also **-A**) |
| **-e** | create trails for all errors encountered (default is first one only) |
| **-f** | add weak fairness (to **-a** or **-l**) |
| **-hN** | choose another hash-function, with *N*: 1..32 (defaults to 1) |
| **-i** | search for shortest path to error (causes an increase of complexity) |
| **-l** | like **-i**, but approximate and faster |
| **-J** | reverse the evaluation order of nested unless statements (to conform to the one used in Java) |
| **-kN** | set the number of hashfunctions used in bitstate hashing mode to *N* (requires compilation with **-DBITSTATE**) The default is k=2. This option was introduced in version 4.2.0. |
| **-l** | find non-progress cycles (requires compilation with **-DNP**) |
| **-mN** | set max search depth to *N* steps (default *N*=10000) |
| **-n** | no listing of unreached states at the end of the run |
| **-q** | require empty channels in valid endstates |
| **-r, -P, -C** | play back error trail with embedded C code statements |
| **-s** | use 1-bit hashing (default is 2-bit hashing, assumes compilation **-DBITSTATE**). In version 4.2.0 and later, the option -s is equivalent to -k1. |
| **-V** | print *Spin* version number and stop |
| **-wN** | use ahashtable of 2^*N* entries(defaults to **-w***18*) |

POLITECNICO
MILANO 1863

# Theoretical background

# Search Algorithms

SPIN uses simple algorithms to verify Promela model correctness, primarily based on **Depth-First Search** (**DFS**).

While basic algorithms are straightforward, their memory and runtime efficiency are critical for large-scale problems.
SPIN's strength lies in its optimization options to handle complex systems effectively.

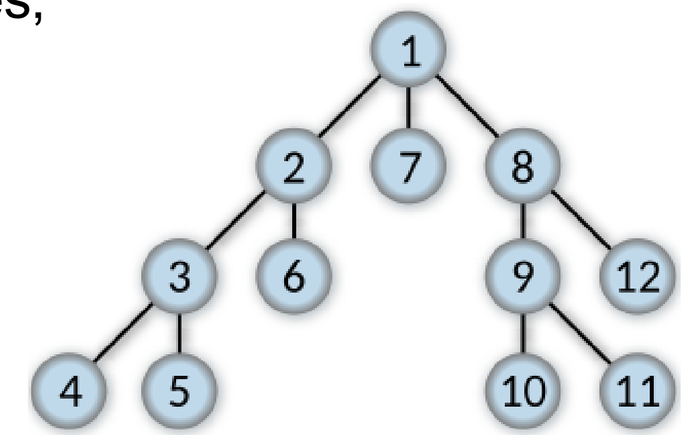
The following slides focus on the core DFS-based search method, while optimization techniques are covered later.

We start with the definition of a DFS algorithm, which we can then extend to perform the types of functions we need for systems verification.

# Search Algorithms: Depth-First Search (DFS)

It systematically explores the state space to **verify properties** like safety, liveness, and ω-properties, **detecting errors** such as deadlocks or **assertion violations**.

- **Initialization:**
  - Starts at the Promela model's initial state (e.g., initial variable values, process states).
  - Uses a stack to track the exploration path and a hash table to mark visited states.
- **Exploration:**
  - Selects an executable transition (e.g., a Promela statement like x = 1 or message send).
  - Computes the resulting new state and checks if it's unvisited.
  - Pushes the new state to the stack and recursively explores it
  - If no transitions are possible or the state is visited, backtracks to the last unexplored state on the stack and tries another transition.




A tree labeled by the order in which DFS expands its nodes

# Search Algorithms: Depth-First Search (DFS)

- **Property Checking:**
  - Safety: In each state, checks for assertion violations, deadlocks, or invalid end states (using **./pan -E**)
  - Liveness/ω-properties: For LTL or never claims, detects acceptance cycles (infinite loops through accepting states) using **./pan -a**, ensuring properties like "*infinitely often p*" hold.
  - Combines model states with never claim states for LTL/omega-property verification.
- **Error Handling:**
  - If a violation occurs (e.g., assertion failure, deadlock, or LTL violation), DFS stops and outputs a counterexample trail (sequence of states/transitions).
  - Trails are replayed in SPIN's simulator (**spin -t -p file.pml**) for debugging.

It's implemented by default in the Pan verifier.

# Search Algorithms: Depth-First Search (DFS)

The algorithm is a DFS pseudocode that explores a state space starting from state **s**.

It:

- Checks if *s* has an error; if so, reports it.
- Adds *s* to **Statespace** to mark it as visited.
- For each successor **t** of *s*, if *t* is not in *Statespace*, recursively calls **dfs(t)**.
- Continues until all reachable states are explored or an error is found.

The *Statespace* is a **stack** which only stores states.

```
proc dfs(s)
    if error(s) then report error fi
    add s to Statespace
    for each successor t of s do
        if t not in Statespace then dfs(t) fi
    od
end
```

# Search Algorithms: Nested Depth-First Search (Nested DFS)

Unlike standard DFS, Nested DFS is designed to **detect acceptance cycles** (infinite loops through accepting states), crucial for verifying properties like "*something happens infinitely often*".
It builds on DFS, adding a second search phase to ensure efficient cycle detection.

- **Initialization:**
  - Starts at the Promela model's initial state, combining model states with never claim states
  - Uses two stacks: one for the primary DFS and one for the nested DFS, plus a hash table for visited states.
- **Primary DFS:**
  - Follows standard DFS: selects an executable transition (e.g., Promela statement like x = 1 or message send), computes the new state, and pushes it to the primary stack if unvisited.
  - Explores deeply along one path, backtracking to unexplored states when reaching a visited state or dead-end.
  - Identifies accepting states (states in the never claim marked as accepting, indicating potential liveness violations).

# Search Algorithms: Nested Depth-First Search (Nested DFS)

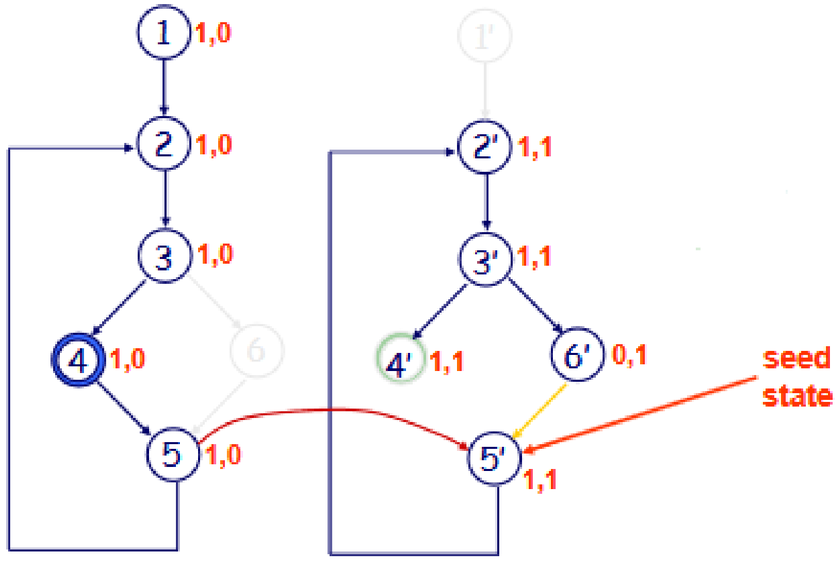
- **Secondary DFS:**
  - When an accepting state is reached in the primary DFS, a second DFS is launched from that state to find a cycle (a path back to the same accepting state).
  - The nested DFS explores all possible transitions from the accepting state, using a separate stack, to detect if the state is reachable again, forming an acceptance cycle.
  - If a *cycle* is found, it indicates a liveness violation (e.g., an infinite path where a property like "*infinitely often p*" fails).
- **Property Checking:**
  - Safety: Handled by primary DFS, checking assertions (assert(condition)), deadlocks, or invalid end states, similar to standard DFS.
  - Liveness/ω-properties: Focuses on detecting acceptance cycles for LTL or never claims, ensuring properties like "*p occurs infinitely often*" or fairness conditions.
- **Error Handling:**
  - If an acceptance cycle is found (liveness violation) or a safety violation occurs, outputs a counterexample trail (state/transition sequence).
  - Trails are replayed via **spin -t -p file.pml** for debugging, showing the cycle or error path.

# Search Algorithms: Nested Depth-First Search (Nested DFS)

The DFS procedure described before can be adapted for **model checking** by using the reachability graph (partially represented in *Statespace*) to form a Büchi automaton with acceptance conditions from a property automaton. Model checking reduces to *detecting accepting states in strongly connected components* (cycles).

The *nested DFS* starts a second DFS when retracting to an accepting state to find a cycle; if none is found, the first DFS resumes.

Adding two bits per state in *Statespace* separates the searches without doubling its size, though search time may double if no cycles exist.




```
proc dfs(s)
    if error(s) then report error fi
    add {s,0} to Statespace
    for each successor t of s do
        if {t,0} not in Statespace then dfs(t) fi
    od
    if accepting(s) then seed:=s; ndfs(s) fi
end

proc ndfs(s) /* the nested search */
    add {s,1} to Statespace
    for each successor t of s do
        if {t,1} not in Statespace then ndfs(t) fi
        else if t==seed then report cycle fi
    od
end
```

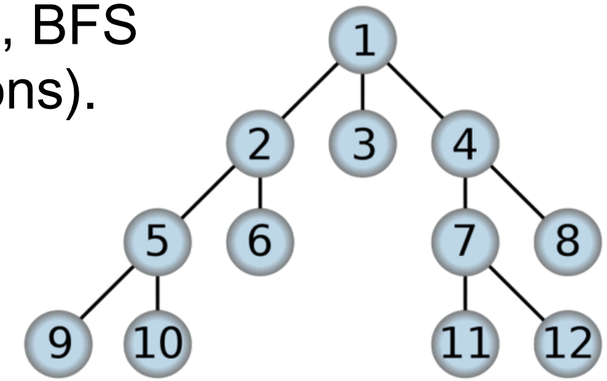
# Search Algorithms: Breadth-First Search (BFS)

BFS in SPIN explores states level-by-level using a queue, finding shorter counterexample paths but requiring more memory and being less efficient for acceptance cycles, but better suited for shallow errors.

- **Initialization:**
  - Starts at the Promela model's initial state (e.g., initial variable values, process states).
  - Uses a queue to store states to explore and a hash table to track visited states.
- **Exploration:**
  - Dequeues the next state from the front of the queue.
  - For each executable transition (e.g., Promela statements like x = 1 or message sends), computes the resulting new state.
  - If the new state is unvisited, marks it as visited and enqueues it at the back of the queue.
  - Processes all states at the current "level" (distance from the initial state) before moving to the next level, ensuring states are explored in order of increasing depth.
  - Continues until the queue is empty (all reachable states explored) or an error is found.

# Search Algorithms: Breadth-First Search (BFS)

- **Property Checking:**
  - Safety: In each state, checks for assertion violations (assert(condition)), deadlocks, or invalid end states (using **./pan -E**).
  - Liveness/ω-properties: For LTL or never claims, verifies acceptance cycles (infinite loops through accepting states) using **./pan -a**, though BFS is less common for cycle detection due to its level-by-level nature.
  - Synchronizes model states with never claim states for LTL/ω-property verification.
- **Error Handling:**
  - If a violation occurs (e.g., assertion failure, deadlock, or LTL violation), BFS stops and outputs a counterexample trail (sequence of states/transitions).
  - Trails are replayed in SPIN's simulator (**spin -t -p file.pml**) for debugging, showing variable values and transitions.




A tree labeled by the order in which BFS expands its nodes

It can be enabled in the Pan verifier (generated via **spin -a file.pml**) using specific compilation flags (e.g., **gcc -DBFS -o pan pan.c**).

# Search Algorithms: Breadth-First Search (BFS)

Consider a Finite State Automata:

***A* = (S, s0, L, T, F)**

The BFS algorithm starts by adding the initial state to the *Statespace* V and **Queue D**, an ordered set of states replacing the search *stack* employed in DFS.
It follows a FIFO policy, iteratively dequeuing a state *s* from the head, processing its transitions in (*A*.T), and enqueuing unvisited successor states (*s*') to the tail.
The search continues until the queue is empty, building V with all reachable states while exploring the state space level-by-level.

```
Queue D = {}
Statespace V = {}
Start()
{
    Add_Statespace(V, A.s0)
    Add_Queue(D, A.s0)
    Search()
}

Search()
{
    s = Del_Queue(D)
    for each (s, l, s') ∈ A. T
        if In_Statespace(V, s') == false
        {   Add_Statespace(V, s')
            Add_Queue(D, s')
            Search()
        }
}
```

# LTL formulae → Büchi Automata

Converts LTL formulae into Büchi automata for verification.

- **Key concepts:**
  - LTL formulae express safety/liveness.
  - SPIN translates LTL to Büchi automata using PROMELA syntax.
- **How it works:**
  - Normalizes formula (negations to atomic propositions).
  - Creates initial state with formula; computes states recursively.
  - Splits states based on subformulas and operators
  - Marks accepting states by “until” subformulas.
- **Automaton Structure:**
  - States: Initial (T0, non-accepting) + accepting (e.g., accept).
  - Transitions: Conditional on propositional formulae, with nondeterministic choices (do..od).

Perfect for verifying temporal properties in concurrent systems.

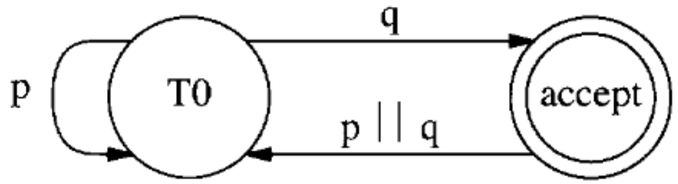
- **Benefits:**
  - Automation: built-in algorithm enhances SPIN’s LTL checking usability.
  - Accuracy: removes user errors from manual conversion.

# LTL formulae ⟶ Büchi Automata

“It is always guaranteed that **p** remains *true* at least until *q* becomes *true*”

```
$ spin -f "[](p U q)"

never {    /* [](p U q) */
T0_init:
        do
        :: ((q)) -> goto accept_S9
        :: ((p)) -> goto T0_init
        od;
accept_S9:
        do
        :: ((((p)) || ((q)))) -> goto T0_init
        od;
}
```




Büchi automata for the LTL formula **[ ] (p U q)**

```
$ spin -f "LTL_formula"
```

## NEVER CLAIMS

A **never claim** is essentially an automaton (a Büchi automaton) that represents the negation of the LTL formula (**¬ φ**).

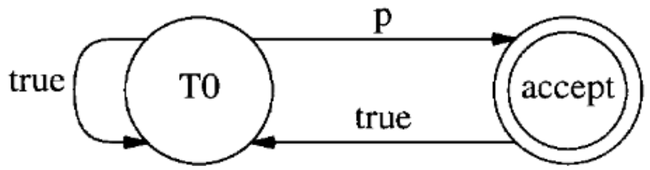
This automaton captures all behaviors that violate the property.

**Acceptance cycles** are infinite loops in the product automaton (system models x never claim) that repeatedly visits an accepting state of the never claim. Detected via nested DSF, they indicate a violation, providing a counterexample for liveness properties

“It is guaranteed that eventually **p** will become *true* at least once more”

```
$ spin -f "[]<>p"

never  {    /* []<>p */
T0_init:
        do
        :: ((p)) -> goto accept_S9
        :: (1) -> goto T0_init
        od;
accept_S9:
        do
        :: (1) -> goto T0_init
        od;
}
```




Büchi automata for the LTL formula **[ ] <> p**

# On-the-Fly LTL Verification

Performs LTL verification efficiently using Nested DFS on automata.

- **Mechanism:**
  - Builds product automaton B × (M1 × M2 ... Mk) on-the-fly:
    - B: automaton from negated LTL formula.
    - M1 × M2 ... Mk: asynchronous product of system models.
    - ×: Synchronous product (B × ...).
  - Uses nested DFS in a single pass to detect accepting ω-runs.
- **How It Works:**
  - Stops if an accepting ω-run (violation proof) is found.
  - Completes full product if no violation; B constrains search to relevant executions.
- **Challenges addressed:**
  - Memory inefficiency: Only builds necessary parts of the product if an error is found.
  - Computation cost: B constrains the product, often making it cheaper than computing M1×..Mk.

Enabled in SPIN for LTL checking (**spin -a -f "LTL_formula" model.pml**; **./pan -a**)

# Adding Fairness

Ensures fair execution: prevents infinite loops in one process while others starve, modeling realistic behavior where all enabled processes eventually progress.

- **Unconditional** Fairness: “Every process gets its turn infinitely often.”
- **Strong** Fairness: “Every process that is enabled infinitely often gets its turn infinitely often.”
- **Weak** Fairness: “Every process that is continuously enabled from a certain time instant on gets its turn infinitely often.” Supported in SPIN.

- **How It Works:**
  - Modifies nested DFS to search for acceptance cycles with *weak* fairness enabled.
  - Tracks process execution: A process must not be indefinitely ignored if it’s enabled
  - Optimization: Stores one state copy with bits overhead, not full copies; bits mark state graph visits.
  - Nested DFS starts from the last state graph copy, ensuring a fair cycle exists if an accepting state is revisited.
- **Challenges addressed:**
  - Unfair cycles: Avoids cycles restricted to one process (detects liveness errors).
  - Memory overhead: Reduces memory usage with bit-based tracking.

Enabled in the Pan verifier using specific execution flag (e.g., **./pan -a -f** )

# Search Optimization

SPIN's explicit state verification algorithms are straightforward, but scaling them for large problems is challenging.

Optimization techniques aim to:

- **Reduce States Searched:**
  - *Partial Order Reduction*: Limits exploration of equivalent execution orders.
  - *Statement Merging*: Combines statements to minimize state transitions.
- **Reduce Memory Usage:**
  - *Lossless Compression*: Methods like *hash-compact*, *collapse compression*, and *minimized automaton representation* save memory while preserving exhaustive verification, often at the cost of runtime.
  - *Lossy Compression*: *Bitstate hashing* aggressively reduces memory and runtime but sacrifices coverage.

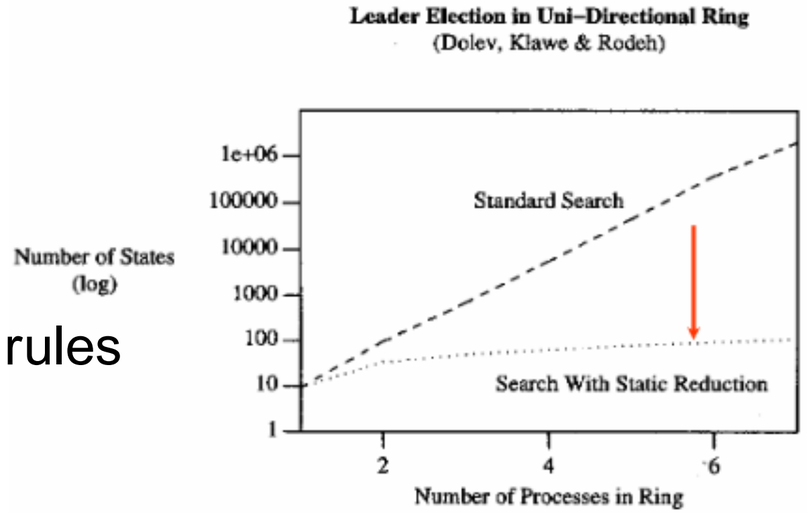
These techniques enhance SPIN's ability to handle large-scale verification efficiently.

# Search Optimization: Partial Order Reduction

Reduces the number of reachable states explored in SPIN verification, minimizing state space size. It's lossless.

Partial Order: orders only dependent actions; independent ones are equivalent.

- **How It Works:**
  - Exploits LTL's insensitivity to the order of independent, concurrent events in DFS.
  - Generates a reduced state space with representative execution sequences (paths) for a given property, instead of all sequences.
  - Uses a static reduction technique: Identifies applicable reduction rules before verification, avoiding runtime overhead.
- **Challenges addressed:**
  - State explosion: Reduces exponential growth to linear; typical savings 10-90% in memory/runtime.
  - Runtime overhead: Static method eliminates past performance issues of partial order reduction.

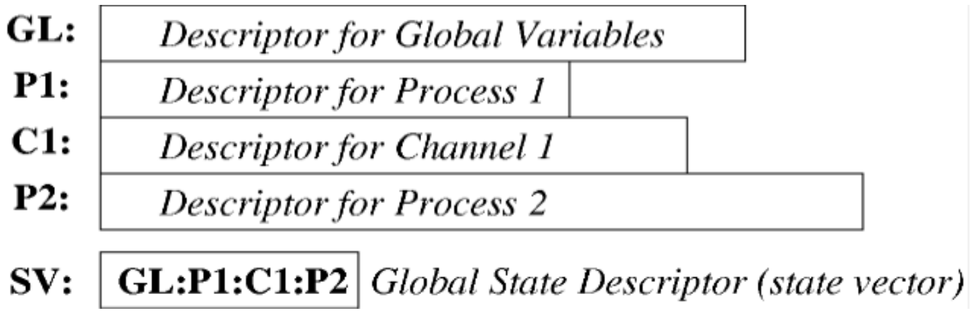



Enabled by default in SPIN verification; no specific flag required (**gcc -o pan pan.c**).

# Search Optimization: Collapse Compression

Reduces memory usage by compressing global state descriptors in SPIN verification. It's a lossless technique.

- **How It Works:**
  - Splits global state descriptors into components: global data (variables, channels, length field) and per-process (control state, local variables).
  - Assigns unique indices to components, stored in tables, forming a compressed descriptor.
  - Uses 2 extra bits per index (up to 4 bytes) for dynamic sizing; stores widths and lengths to ensure accuracy.




- **Challenges addressed:**
  - State Space Explosion: Growth from combining local states (process control states, variable values).
  - Inefficient Storage: Avoids redundant full state replication in Statespace.

It can be enabled in the Pan verifier (generated via spin -a file.pml) using specific compilation flags (e.g., gcc -DCOLLAPSE -o pan pan.c).

EFFECT OF COMPRESSION

| Type of Run | No. States | Memory (Mb) | Time (sec.) |
|---|---|---|---|
| Standard | 2,435,220 | 156.59 | 107.56 |
| Compressed | 2,435,220 | 59.57 | 123.46 |

# Search Optimization: Minimized Automata Representation

Reduces memory usage by storing reachable states in a minimal DFA instead of a lookup table.

- **How It Works:**
  - Builds a DFA as a finite graph to recognize state descriptors.
  - Interrogates the DFA for each system state during search; updates it for new descriptors.
- **Challenges addressed:**
  - High Memory Use: Achieves exponential memory savings compared to standard methods.
  - Storage Inefficiency: Avoids conventional lookup tables for state matching.

This is a lossless technique.

It can be enabled in the Pan verifier (generated via **spin -a file.pml**) using specific compilation flags (e.g., **gcc -DMA=N -o pan pan.c**).

(N = estimated depth, e.g., state vector size from a normal run)

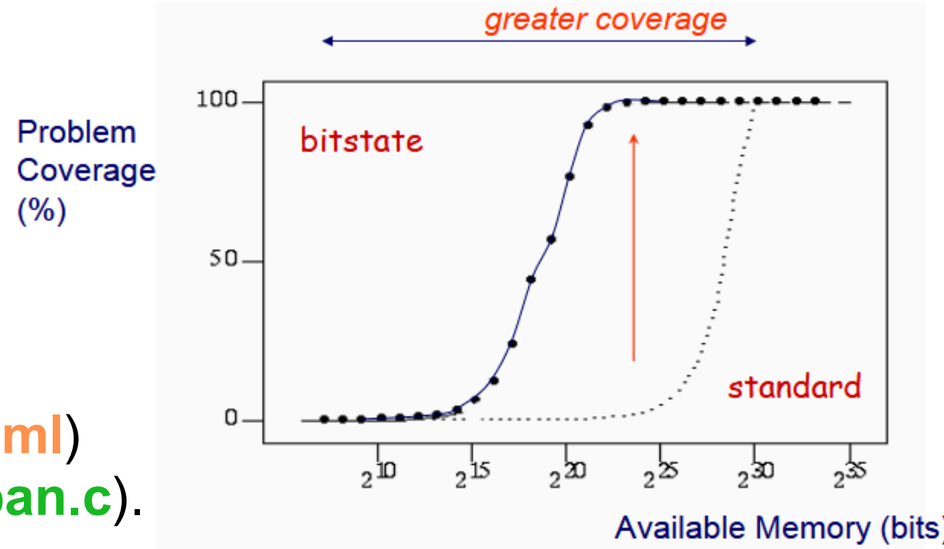
| *Algorithm* | Average Mem. (Mb.) | Average Time (sec.) |
|---|---|---|
| Uncompressed | 24.04 | 12.61 |
| Collapse | 8.48 | 16.94 |
| DFA+Collapse | 4.35 | 78.17 |
| DFA | 3.37 | 61.82 |
| GETS | 3.30 | 65.44 |

Memory and Time Averaged over Fourteen Typical Applications

# Search Optimization: Bitstate Hashing

Enhances state-space coverage verification for large models with limited memory.

- **How it works:**
  - Uses lossy compression: Stores N bits per state (default: 2) in M-byte memory, computing N independent hash values (bit-addresses) of $\log_2$ (8M) bits
  - Sets bits in a bit-array (e.g., 0 to 1); collisions may skip states (no false positives).
  - *Coverage*: Standard: M / (R×S); bitstate: ~100% if M / R ≫ 4.
  - *Control*: Adjust hash-array size for coverage vs. runtime; larger arrays increase coverage but take longer.
- **Challenges addressed:**
  - Limited memory: Outperforms exhaustive runs when M < R×S.
  - State explosion: Achieves higher coverage despite collisions via random truncation.




It can be enabled in the Pan verifier (generated via spin -a file.pml) using specific compilation flags (e.g., gcc -DBITSTATE -o pan pan.c).

# Search Optimization: Hash-Compact

Reduces memory usage by storing compact 64-bit hash values instead of full states, simulating a large memory size (e.g., $2^{64}$ bits). It's lossless.

- **How It Works:**
  - Computes a single 64-bit hash value (range 0 to $2^{64}$-1) for each state using one hash function.
  - Stores hash values in a regular hash table, not full states, with a collision probability near $10^{-57}$ for $10^7$ states, aiming for 100% coverage.
  - Memory usage: $10^7$ 64-bit numbers use ~$10^9$ bits, adjustable based on **m / r** ratio (m = memory, r = states, S = size of a single state descriptor).
  - Optimal when $r \times 64 \leq m \leq r \times S$, though *r* is often unknown pre-verification.
- **Challenges addressed:**
  - High memory demands: Uses less memory than exhaustive storage.
  - State explosion: Simulates large memory ($2^{64}$ bits) for better coverage with limited resources.

It can be enabled in the Pan verifier (generated via **spin -a file.pml**) using specific compilation flags (e.g., **gcc -DHC4 -o pan pan.c**).

# Search Optimization: Bounded Context Switching

Finds simpler, more practical counterexamples by limiting context switches in SPIN verification. It's a lossless technique.

- **How It Works:**
  - Extends DFS: Explores states, but bounds unforced context switches to N.
  - Tracks context switches per state; revisits states if reached with fewer switches up to N×P , P=processes).
  - Non-preemptive switches (forced) are not counted; stores states with minimum switches and process IDs.
  - Supports safety properties; increases complexity (path-based, not state-based).
- **Challenges addressed:**
  - Complex counterexamples: Reduces context switches for easier debugging (vs. standard DFS).
  - Memory inefficiency: Less memory than BFS, though requires revisiting states.

It can be enabled in the Pan verifier (generated via **spin -a file.pml**) using specific compilation flags (e.g., **gcc -DBCS -o pan pan.c**).

# Model Extraction & Abstraction

Simplifiy verification of distributed software by generating finite models from complex applications.

- **How It Works:**
  - Abstraction: Reduces software to FSA by omitting irrelevant details, while preserving properties of interest.
  - Extraction: Parser extracts a control flow skeleton; a lookup table defines which statements to omit, abstract (e.g., non-deterministic choices for function calls), or preserve as transitions in an FSA.
  - Embedded C Code: SPIN supports C code integration for abstraction without extraction, allowing verification of implementation-level code with user-defined abstraction functions.
- **Challenges addressed:**
  - Undecidability: Makes verification feasible by reducing software to finite models.
  - Complexity: Automates abstraction/extraction (*e.g.*, verifies 300 versions of call processing software, finding 75 critical errors (race conditions)).

Necessary due to undecidable software properties; enables early detection of subtle race conditions.

POLITECNICO
MILANO 1863

# PROMELA

# PROMELA (Process/Protocol Meta Language)

It's SPIN's **modeling language** for specifying distributed systems

- **Not a programming language**
  - Resembles the C-language but is not a programming language,
  - It's rather a meta-language for building verification models.
  - The design of Promela is focused on the interaction among processes at the system level.

- **Non-deterministic**
  - Optimal for the formal verification of the system behaviour exploring all possible paths (especially the worst), and analyze concurrent processes.

- **To do that it provides:**
  - Non-deterministic control structures,
  - Primitives for process creation and for interprocess communication.

# PROMELA elements

- Processes
- Variables and data types
- Control flow constructs
- Channel communication
- Verification

# Processes

- Programs in PROMELA are composed of a set of processes
- To declare a process and describe its behavior we use the **proctype** keyword

```
proctype Pname() {
    //code to execute
}
```

- The process can have a list of parameters
- The proctype definition only declares process behavior, it does not execute it.
- Each process has its own local process counter **_pid**

# Processes

So how we initiate/run these processes?

- **Active keyword: the process instance starts as soon as the model begins**

```
active [n] proctype Pname() {
    //code to execute
}
```

to spawn more instances of the same process we add [n] with n the number of copies, if wew omit it is implied 1

- **Run keyword: we can use run Pname() inside any process to start its executions**

# Processes - *init*

- It’s a special type of process which runs automatically as model begins (if declared)
- Only one instance of init exists
- It’s like the “main” process of promela
- Doesn’t have parameters
- Used to set up the system (run other processes, initialize variables)

```
init {
    run Pname();
}
```

# Processes

Who start first?

- The **init** process *always starts first*, in particular its first action.
  Then all actions of started processes (including init and active ones) are treated equally and scheduled **non-deterministically**
- All processes are scheduled concurrently, and SPIN explores all possible **interleavings**.
- Each process may have several different possible actions enabled at each point of execution: only one choice is made (*non-deterministically*).
- if we want to execute actions without interleaving we must use atomic{} or d_step{} keywords

# Processes - Example

```
active proctype A() {
    printf("A\n");
}

proctype B() {
    printf("B\n");
}

init {
    run B();
    printf("init\n");
}
```

If we run the test we can obtain various output (3!):

- **init, A, B**
- **init, B, A**
- **A, B, init**
- **A, init, B**
- **B, A, init**
- **B, init, A**

**run B()** is runned first enabling process B → 3 concurrent process (each one with one action)

# Variables and Data types

| Type | Typical Range |
|---|---|
| bit | $0, 1$ |
| bool | *false*, *true* |
| byte | $0..255$ |
| chan | $1..255$ |
| mtype | $1..255$ |
| pid | $0..255$ |
| short | $-2^{15} .. 2^{15}-1$ |
| int | $-2^{31} .. 2^{31}-1$ |
| unsigned | $0 .. 2^{n}-1$ |

- All variables are initialized by default to 0
- There are no floats and no strings
- A byte can be printed as a character with the **%c** format specifier
- Variables can be local or global (as in C)
- **mtype** example:

  *mtype = { RED, GREEN, BLUE };*
  *mtype color = RED;*

# Control flow constructs

While the syntax and semantics of expressions in PROMELA are taken from C-like languages, the control statements are taken from a formalism called guarded commands invented by E.W. Dijkstra.
This formalism is particularly well suited for expressing nondeterminism.

There are five control statements:

- **sequence**
- **selection**
- **repetition**
- **jump**
- **unless**

# Control flow constructs - Selection

## If-statement


```
if
:: condition1 -> action1
:: condition2 -> action2
....
:: else → action3
fi
```

- One sequence of the list will be executed
- *guards* could be mutually exclusive (but it's not a must). If more than one guard is executable, one of the corresponding sequences is selected **non-deterministically**

- If all guards are not executable, the process will block until one of them can be selected. Needs to be unblocked by a concurrent process.
- It's possible to add an else statement: if and only if all the other guards evaluate to false, the statements following the else will be executed.

# Non-determinism - Example

```
active proctype P() {
    int a = 5, b = 5;
    int max;
    int branch;
    if
    :: a >= b -> max = a; branch = 1;
    :: b >= a -> max = b; branch = 2;
    fi;
    printf("The maximum of %d and %d = %d
    by branch %d\n", a, b, max, branch);
}
```

One branch will be taken at random,
so if we run multiple times the printf will
print sometimes "*...by branch 1*"
and other times "*...by branch 2*"

# Control flow constructs - Repetition

## Do statement

```
do
:: condition1 -> action1;
:: condition2 -> action2;
....
:: condition -> break;
od
```

- Only one of the guard can be taken each time we enter the structure (non-deterministically if more are **true**).
- If at least one guard is *true* its instructions are **executed**, otherwise it **blocks** *until* one guard becomes *true*.
- If the **break** statement is encountered, it transfers the control to the instruction that immediately follows the repetition structure.
  Otherwise the structure is repeated

# Example: sum of N numbers

```
#define N 10

active proctype P() {
    int sum = 0;
    byte i = 1;

    do
    :: i > N -> break
    :: else -> sum = sum + i;
                i++
    od;
    printf("The sum of  the first %d numbers = %d\n", N, sum);
}
```

**in Promela we don’t have directly a for statement but we can build it with the do statement**

# Control flow constructs - Jump

Like in C, PROMELA contains a **goto**-statement that causes control to jump to a label, which is an identifier followed by a (single) colon.

**goto** is powerful but can be misused: gives you the freedom to jump anywhere within the same scope. So is good practice to use it only if necessary.

For example, **goto** can be used to exit a loop, but it's preferred to use **break** since it is more structured and doesn't require a label.

# Control flow constructs - Unless

```
bool pressed = false;

active proctype Device() {
    byte i = 1;

    do
    :: i <= 10 ->
        printf("Counting: %d\n", i);
        i++
    od
    unless
    :: pressed ->
        printf("Button pressed! Interrupted at %d\n", i)
}
```

It's a special statement useful for modeling interruptions or exceptions.

After the unless statement we have one or more guards which form an escape block.
If any condition of the escape block becomes *true* the normal execution is **blocked** and it's executed instead the action following the condition.

# Channel communication

Communication between process could be done through:

- **global variables**
- **channels**

A *channel* is a FIFO message queue used to send and receive messages between processes.

*chan chname = [capacity] of {type1, type2, ...};*

**how many messages can hold the queue**
**if capacity > 0 → buffered channels**
**if capacity = 0 → synchronous (rendezvous) channels**

**types of values carried in the message**

# Channel communication - primitives

- **Send** a message: *chname!expr1, expr2,...*

  **The process blocks if the channel is full.**

- **Recive** a message: *chname?var1, var2,...*

**The process blocks if the channel is empty.**

- There are also built-in functions that allow you to check the status of a channel: **len, empty, nempty, full, nfull.**
  - **len(chname) → #messages in the queue**

# Verification

- **assert:** stop the execution if the condition in not true

  *assert(x >= 0); // stops if x < 0*

- **accept: , progess:** labels used to verify specific state goals and check the liveness of the execution

- **LTL formulas:** let us express temporal rules which must be valid over time instead of a single instant. We can use these formulas directly in the LTL property box of the GUI or if we want to use the terminal:

```
spin -f "[ ](x <= 5)"

spin -a model.pml
gcc -o pan pan.c
./pan -n
```

→ **This generates the equivalent never { ... } block to copy to the .pml code**

→ **then compile and verifiy**

# LTL syntax

| Symbol | Name | Meaning / Interpretation |
|---|---|---|
| []p | Always | p holds in all future states |
| <>p | Eventually | p will be true at some point in the future |
| p U q | Until | p holds until q becomes true |
| p V q | Release | q is released when p becomes true. |
| p -> q | Implies | if p is true, then q must be true |
| p <-> q | Equivalence | p and q are logically equivalent |

and also the operators **AND** (&&), **OR** (| |) and **NOT** (!)

POLITECNICO
MILANO 1863

# Extensions

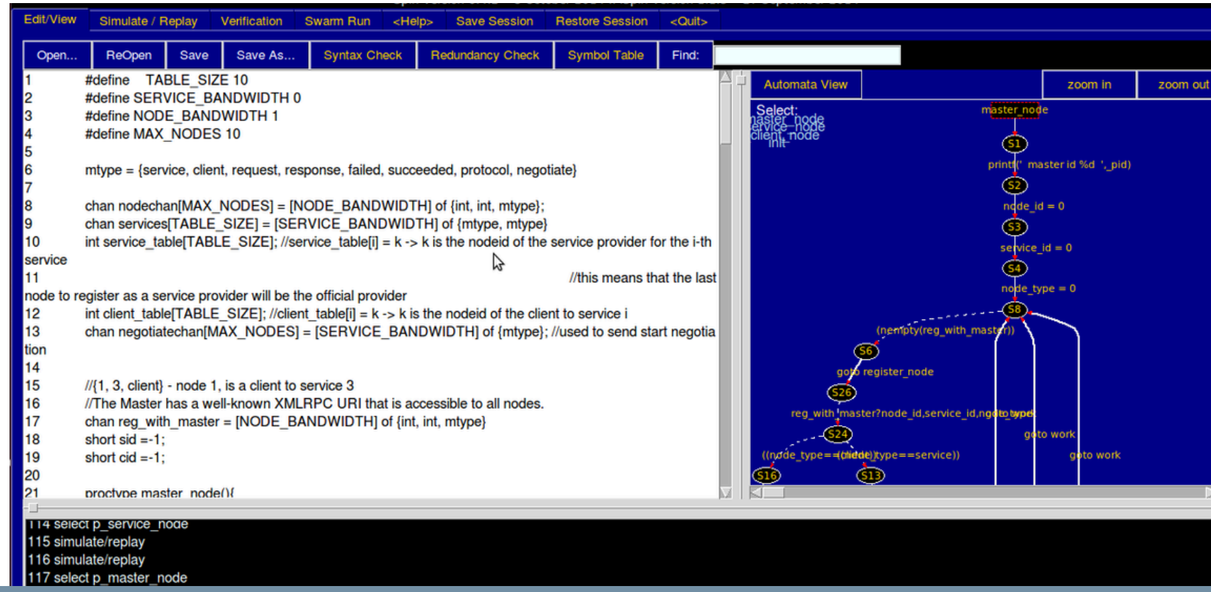
# iSPIN

iSPIN is a Tcl/Tk-based **GUI** for SPIN, designed to verify concurrent and distributed systems.

It offers an intuitive platform for editing, simulating, and analyzing PROMELA models, with syntax highlighting, integrated visualization, automated SPIN option generation, and filtered output formatting.

It also provides interactive simulation control, error trail navigation, customizable verification settings, and LTL property checking to streamline model development and debugging.

iSpin is the modern successor of XSpin.



# iSPIN - Setup

It is assumed that SPIN and related tools are properly installed as previously outlined

## Windows 

1. Download “*ispin.tcl*” file from https://github.com/nimble-code/Spin/tree/master/optional_gui
2. Move it inside the “cygwin/bin” directory

## Linux 

**Prerequisite.** Make sure to have ***dot*** package installed

1. Copy the executable located inside “Spin/optional_gui” into “/usr/bin” to be able to run **ispin** from anywhere

```
$ cp ispin.tcl /usr/bin
```

# Swarm

Swarm is a **verification script generator** *front-end* for SPIN

It generates a script for running multiple small and randomly different SPIN verification jobs in parallel to boost coverage for large models.
It’s designed for cases where exhaustive, bitstate, or hash-compaction verification exceeds memory limits or takes too long (e.g., days/weeks).

Swarm uses parallelism and search diversification to reach its objectives.

Users can use a configuration file specifying available CPUs, memory, time, and optional settings. Swarm then generates a script to run maximum independent jobs within these constraints.

# Swarm - Basic Usage

By default, **swarm** creates a *.swarm* script using the same basename as the SPIN model. The only required parameter is the model file name, though several optional flags can further customize the verification.

```
$ swarm -f model.pml    # generate the script (model.swarm)

$ ./model.swarm         # execute it
```

The easiest way to manage options is with a configuration file. Use the ***-l*** flag to generate a default one—any options set before ***-l*** (like CPU count or max runtime) will be included in it. *swarm.lib* is the config file created.

```
$ swarm -l > swarm-lib
# create configuration file (which can be edited)

$ swarm swarm.lib
# use the settings from this file to generate the script
```

More details can be found in the GithHub repository https://github.com/nimble-code/Swarm

Swarm can be run directly within iSPIN, using the dedicated “Swarm Run” view.

# Swarm - Setup

It is assumed that SPIN and related tools are properly installed as previously outlined

**Prerequisite.** SPIN ≥ *5.2.0*

## Windows

1. Download Swarm GitHub repository
   https://github.com/nimble-code/Swarm
   extract the compressed folder and move it into Cygwin directory
2. Open Cygwin Terminal and navigate to "Swarm/Src" folder
3. Run the following command

```
$ make
```

4. Move the newbuilt "*swarm.exe*" file into "cygwin/bin"

## Linux

1. Clone Swarm GitHub repository
   https://github.com/nimble-code/Swarm

```
$ git clone https://github.com/nimble-code/Swarm.git
```

2. Move into "Swarm/Src" folder and compile the files inside

```
$ cd Swarm/Src

$ make
```

3. Copy the executable into "/usr/bin" to be able to run **swarm** from anywhere

```
$ cp swarm /usr/bin
```

# Modex

Modex is a tool that **mechanically extracts** Promela **verification models** from C code, which SPIN then analyzes to check for logical properties.

It parses C code, applying abstractions from a *.prx* test harness to focus on key behaviors. Abstractions simplify details (e.g., variables to booleans) to prevent state-space explosion. The *.prx* file specifies functions, abstraction rules, and environment (e.g., stubbed inputs), creating a compact Promela model for SPIN verification.

Modex simplifies the complex and error-prone process of manually creating Promela models for intricate C programs, enabling verification of large software systems beyond traditional SPIN's modeling capacities.

# Modex - Basic Usage

**modex** parses the C code in file.c, extracts a Promela verification model using predefined or specified rules, and generates *model* (Promela file) and *_modex_.run* (script for SPIN verification).

```
$ modex code.c

# MODEX Version 2.11 - 3 November 2017
# created: model and _modex_.run
```

There is a script named *verify* in the "Modex/Scripts" directory that can simplify working with Modex. Copy it in the "/bin" directory

**verify** first runs modex to create *model* and *_modex_.run*. It then executes *_modex_.run*, which runs SPIN (spin -a model), compiles the verifier (gcc -o pan pan.c), and checks for errors (./pan), outputting results like assertion failures or state counts. So It combines modex's model generation with SPIN verification.

```
$ verify code.c

#       Extract Model:
#       ==============
# modex code.c
# MODEX Version 2.11 - 3 November 2017
# created: model and _modex_.run
#
#       Compile and Run:
#       ================
# sh _modex_.run
# .....
# .....
```

# Modex - Setup

It is assumed that SPIN and related tools are properly installed as previously outlined

**Prerequisite.** SPIN ≥ *6.4.6*, ***bison*** and ***flex*** packages installed

## Windows

1. Download Modex GitHub repository
   https://github.com/nimble-code/Modex
   extract the compressed folder and move it into Cygwin directory
2. Open Cygwin Terminal and navigate to "Modex/Src" folder
3. Run the following command

```
$ make
```

4. Move the newbuilt "*modex.exe*" file and the into "cygwin/bin"

## Linux

1. Clone Modex GitHub repository
   https://github.com/nimble-code/Modex

```
$ git clone https://github.com/nimble-code/Modex.git
```

2. Move into "Modex/Src" folder and compile the files inside

```
$ cd Modex/Src

$ make
```

3. Copy the executable into "/usr/bin" to be able to run **modex** from anywhere

```
$ cp modex /usr/bin
```

# Other extensions

| | |
|---|---|
| **ETCH** | Standalone type checker for Promela, enhancing SPIN's limited type checking by reconstructing channel types, enabling strong static error detection for faster, more reliable model verification. |
| **jSPIN** | Java-based GUI for the SPIN Model Checker, designed for educational use. It offers a simple interface with configurable menus, toolbar, and text areas, automatically handling SPIN options and formatting output. |
| **SPINSPIDER** | Integrated with jSpin but also standalone, generates graphical state diagrams from SPIN output for PROMELA programs, illustrating concurrent programming properties. |
| **Tau** | Tau (“*Tiny Automata*”) is a simple GUI for experimenting with formal verification and small automata. It uses a background tool called 't2s' (“*tau to spin*”) to translate Tau specifications to Spin models. Find more downloading https://spinroot.com/spin/tau_v1.tar.gz |
| **TopSPIN** | Automatic symmetry reduction tool for SPIN, using group theory to optimize Promela model verification, reducing memory use and often speeding up the process. |

More information and extensions are available at https://spinroot.com/spin/Man/README.html

POLITECNICO
MILANO 1863

# Real-World Applications

# Mission Critical Software

**Scope** The Mars Science Laboratory mission, launched to deploy the Curiosity rover on Mars in 2012, aimed to explore the planet surface, assess its habitability and achieve scientific objectives, relying on meticulously verified software and hardware systems.

**SPIN usage** Spin was used to verify critical multithreaded software components (e.g., boot-control, file system, data management) for concurrency issues like race conditions, employing automated model extraction and manual modeling.

## Conclusions

High-reliability software and hardware design, supported by formal methods and automated model-checking, effectively mitigate risks like race conditions and deadlocks, ensuring spacecraft functionality and remote debugging capability.

# The Toyota Investigation

**Scope** Determine whether Toyota's electronic throttle control system (ETCS) has any design or software issues that could cause unintended acceleration (UA) during normal driving.

**SPIN usage** Formal verification of embedded software logic models to identify potential design flaws, especially in multi-threaded systems. Paired with Swarm, the process was scaled across multiple processors to efficiently analyze large and complex verification tasks in parallel.

## Conclusions

No Toyota vehicle was found to naturally cause large throttle opening UA effects. The ETCS system has safety features to prevent UA, with no electrical failures affecting braking or software defects identified. Pedal assembly failures need specific conditions, and driver awareness is vital for mitigation.

# Flood Control

**Scope** The BOS project builds an automated storm surge barrier near The Hook of Holland to protect western Netherlands from floods. It uses the BOS system to control the barrier, gathering data from sensors and predicting water levels with high reliability.

**SPIN usage** Spin, paired with Promela, validated key design interfaces, leveraging its large state space handling and simplicity, enabling engineers to adopt it easily and ensure design correctness in the project.

## Conclusions

Validation, aimed at confirming system correctness or identifying errors, stops based on objectives: fixed criteria met with no errors, or satisfaction with errors found, balancing effort and error severity.

POLITECNICO
MILANO 1863

# Bibliography

# Bibliography


**Book**

The SPIN Model Checker - Primer and Reference Manual
G.J. Holzmann • 2003 • Addison Wesley

**Papers**

Using Spin
G.J. Holzmann • 1995 • Plan 9 Programmer s Manual Documents

The Model Checker SPIN
G.J. Holzmann • 1997 • IEEE Transactions on Software Engineering

Software Model Checking with SPIN
G.J. Holzmann • 2005 • Advances in Computers

The Application of Promela and Spin in the BOS Project (abstract)
P. Kars • 1996 • University of Twente

Mars code
G.J. Holzmann • 2014 • Communications of the ACM

Toyota Unintended Acceleration Investigation
M.T. Kirsch • 2011 • NASA Engineering and Safety Center

# Bibliography

## Papers


On Nested Depth First Search
G.J. Holzmann et al • 1997 • DIMACS Series in Discrete Mathematics and Theoretical CS

Model Checking with Bounded Context Switching
G.J. Holzmann, M. Florian • 2010 • Formal Aspects of Computing


## Slides


Advanced SPIN Tutorial
T. Ruys, G.J. Holzmann • 2004 • SPIN 2004 Tutorial • Barcelona

Quick Reference
A. Burak Gurdag • 2007 • https://spinroot.com/spin/Man/index.html

The SPIN Model Checker
A. Corradini • 2013 • Metodi di Verifiica del Software • University of Pisa

Formal Methods for Concurrent and Real-Time Systems
P. San Pietro • 2025 • Politecnico di Milano

# Bibliography

**Links**

https://spinroot.com/spin/

https://www.cygwin.com/install.html

https://en.wikipedia.org/wiki/Depth-first_search

https://en.wikipedia.org/wiki/Lamport%27s_bakery_algorithm

**GitHub Repositories**

https://github.com/nimble-code/Spin

https://github.com/nimble-code/Swarm

https://github.com/nimble-code/Modex

POLITECNICO
MILANO 1863

# Demo

# Lamport’s Bakery Algorithm

*Lamport's Bakery Algorithm* ensures threads access a shared resource’s critical section without conflicts, allowing only one thread at a time.

Picture a bakery: each thread grabs a ticket with a number higher than others. The thread with the lowest number—or lowest ID if tied—enters the critical section, like being served by the baker. Threads wait for others to finish picking tickets to avoid errors. After finishing, a thread discards its ticket, letting the next proceed.

This ensures fairness, prevents deadlocks, and guarantees every thread’s turn.

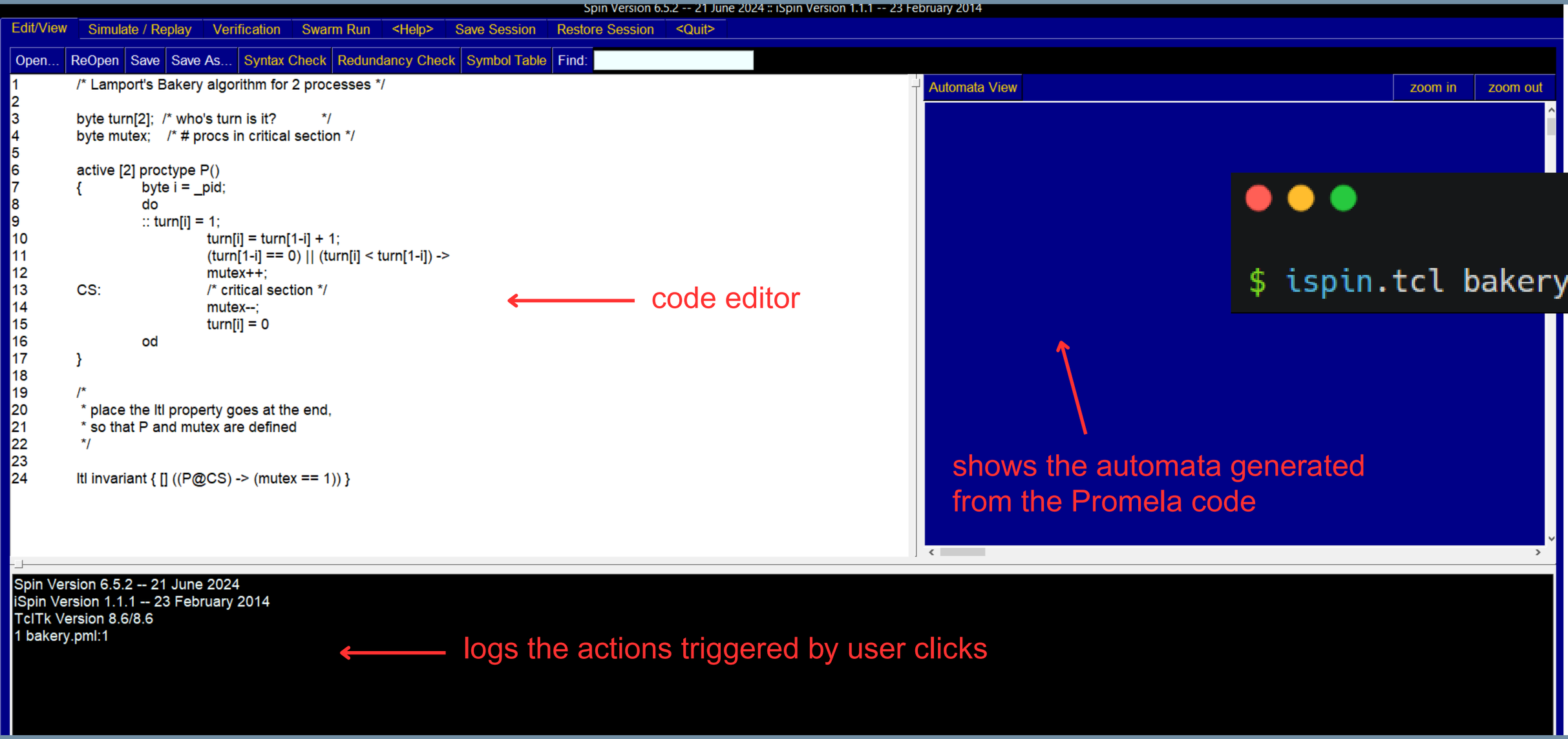
# bakery.pml

```
1       /* Lamport's Bakery algorithm for 2 processes */
2
3       byte turn[2];  /* who's turn is it?          */
4       byte mutex;    /* # procs in critical section */
5
6       active [2] proctype P()
7       {           byte i = _pid;
8                   do
9                   :: turn[i] = 1;
10                              turn[i] = turn[1-i] + 1;
11                              (turn[1-i] == 0) || (turn[i] < turn[1-i]) ->
12                              mutex++;
13      CS:                     /* critical section */
14                              mutex--;
15                              turn[i] = 0
16                  od
17      }
18
19      /*
20       * place the ltl property goes at the end,
21       * so that P and mutex are defined
22       */
23
24      ltl invariant { [] ((P@CS) -> (mutex == 1)) }
```

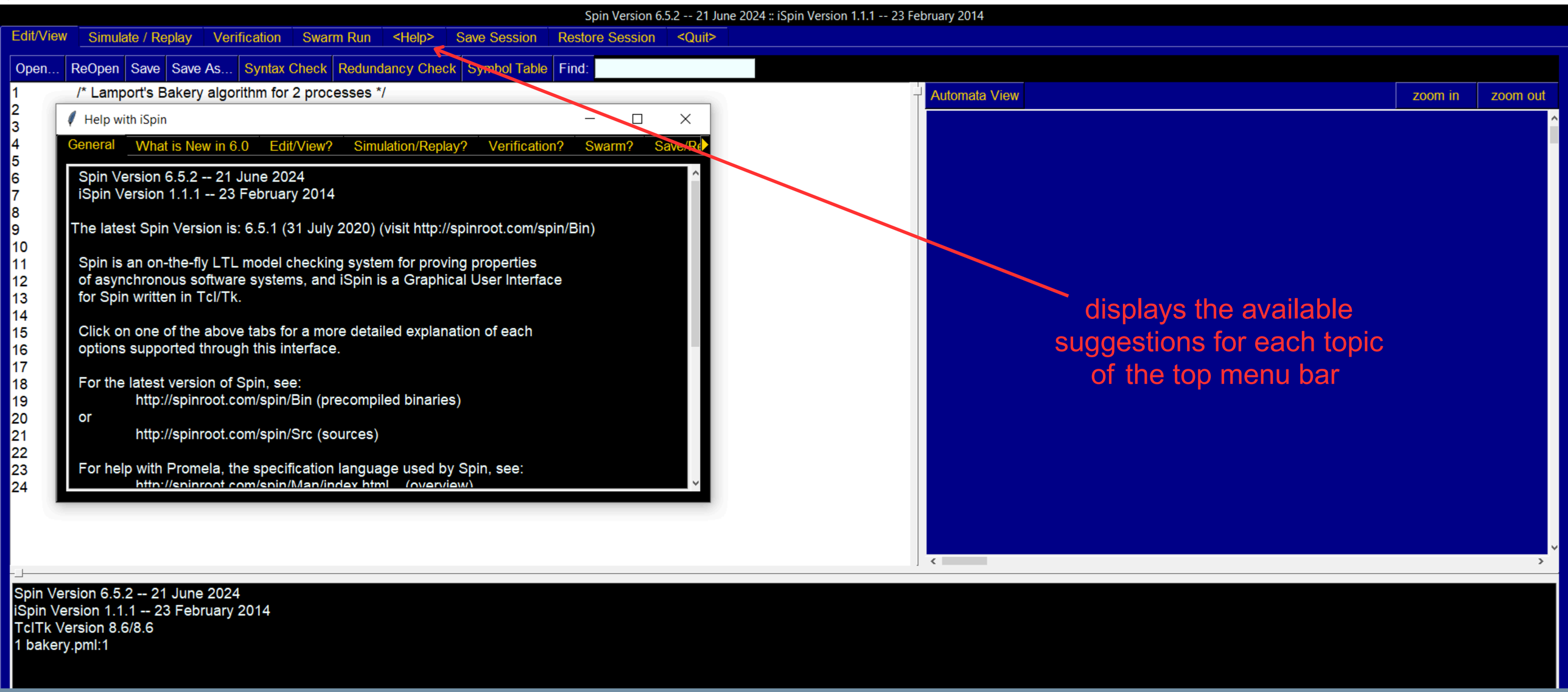
*"Always, if a process is in the critical section, then the variable **mutex** is equal to 1."*

If **mutex** is *different* than 1 while a process is in the critical section, SPIN would detect a violation and provide a counterexample

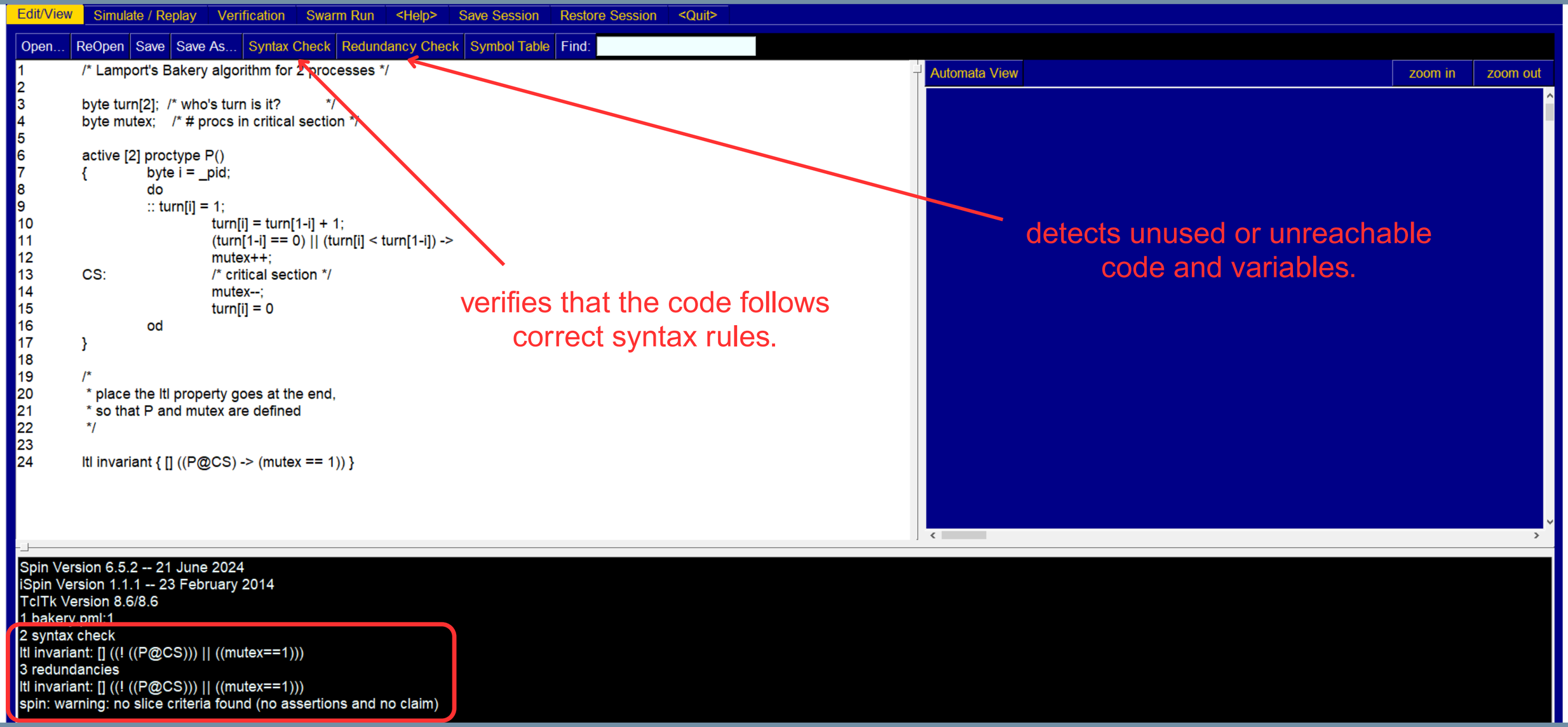
# First look at iSPIN



# First look at iSPIN



# First look at iSPIN

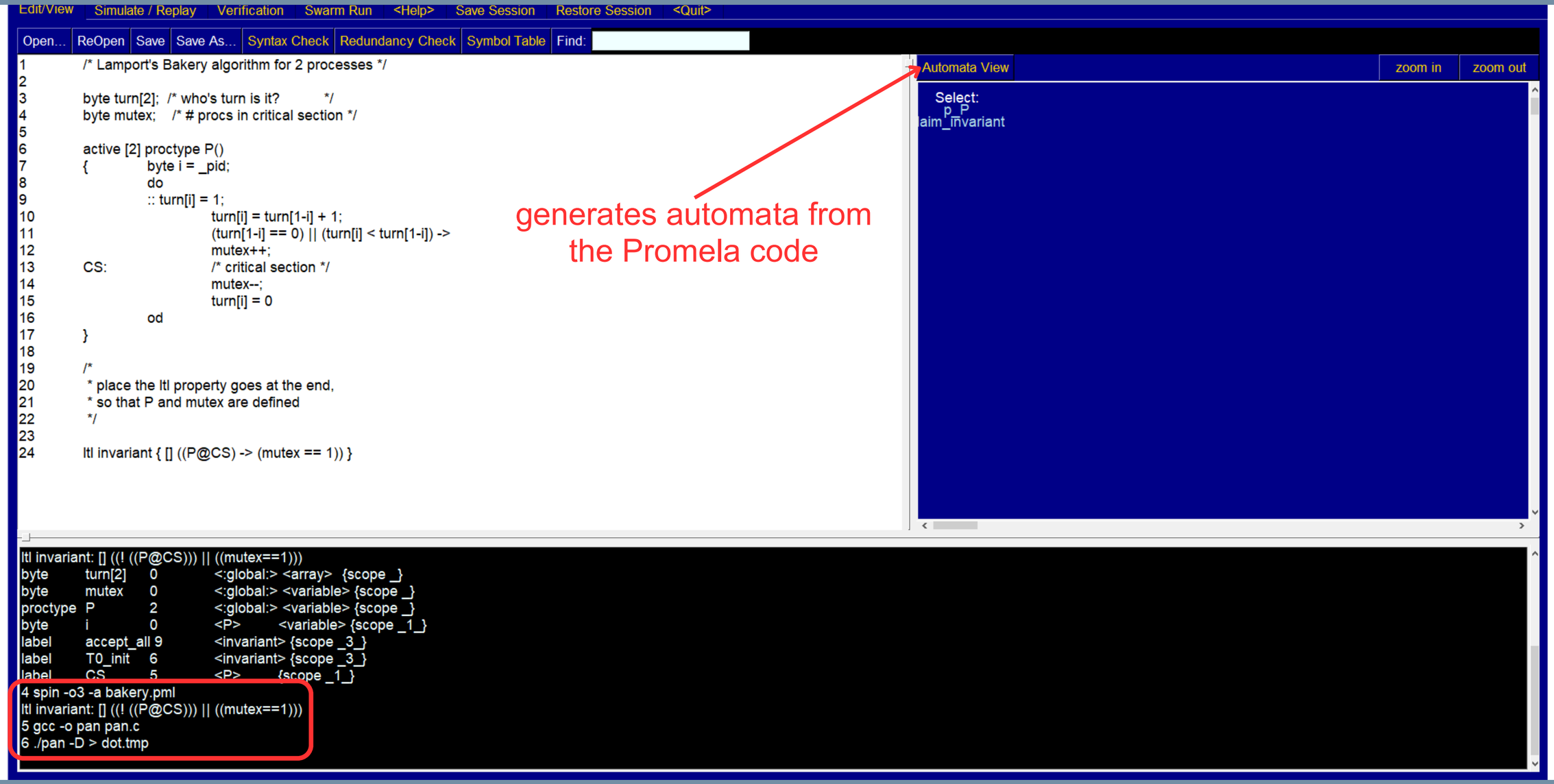



verifies that the code follows correct syntax rules.

detects unused or unreachable code and variables.

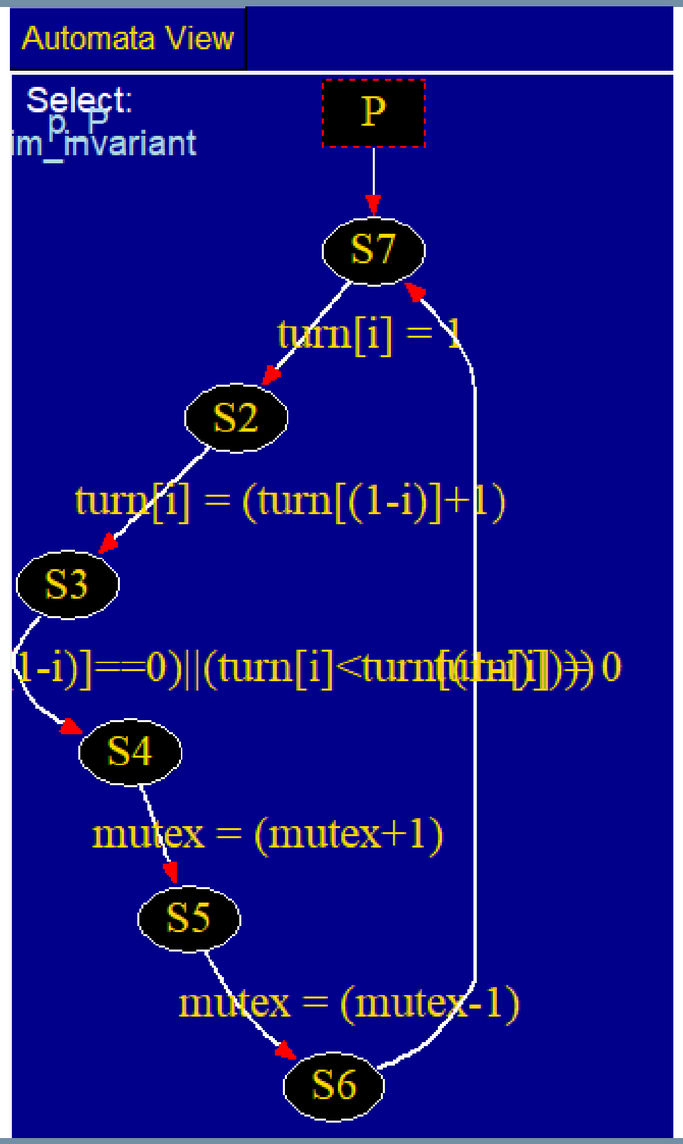
# First look at iSPIN

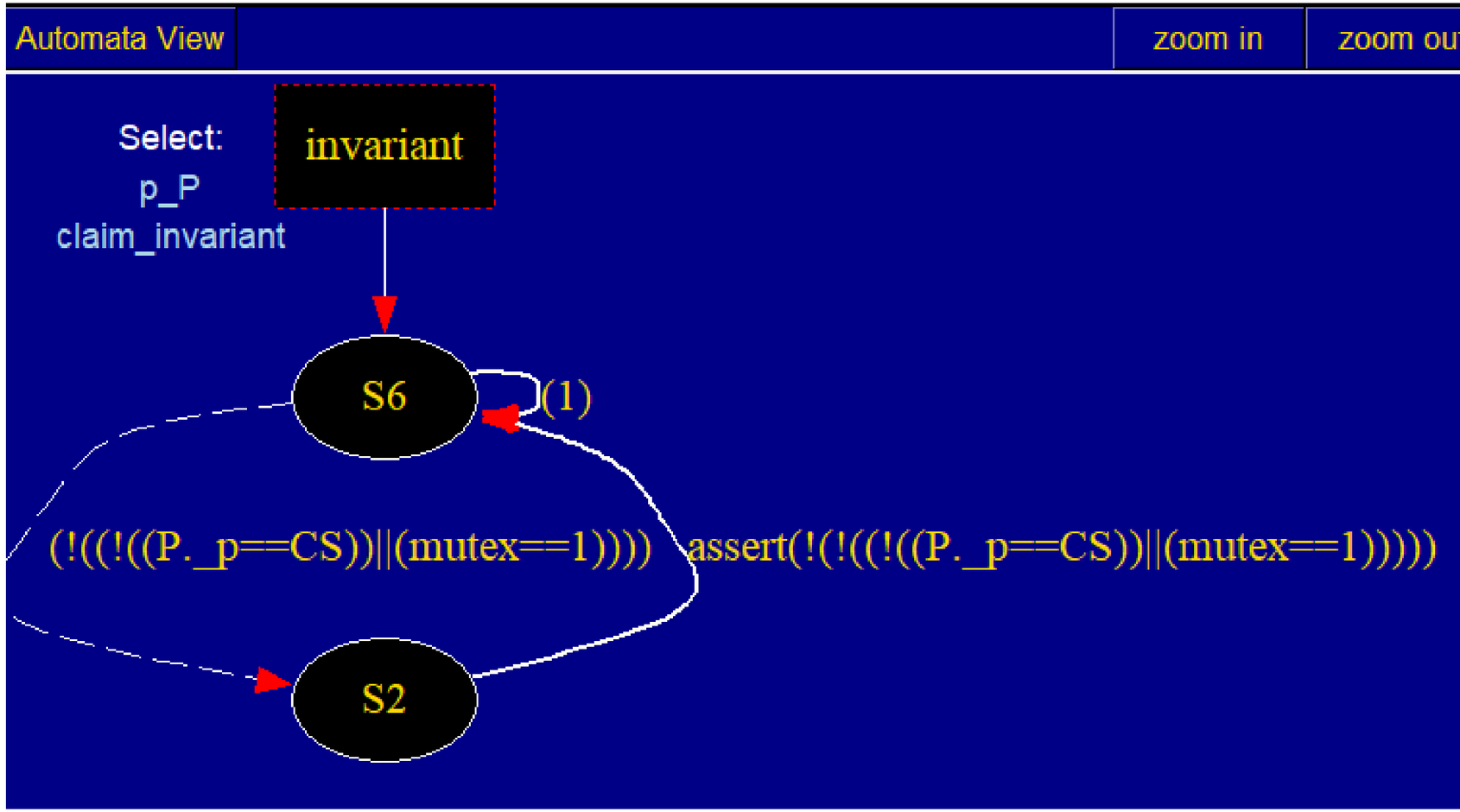
# Automata View



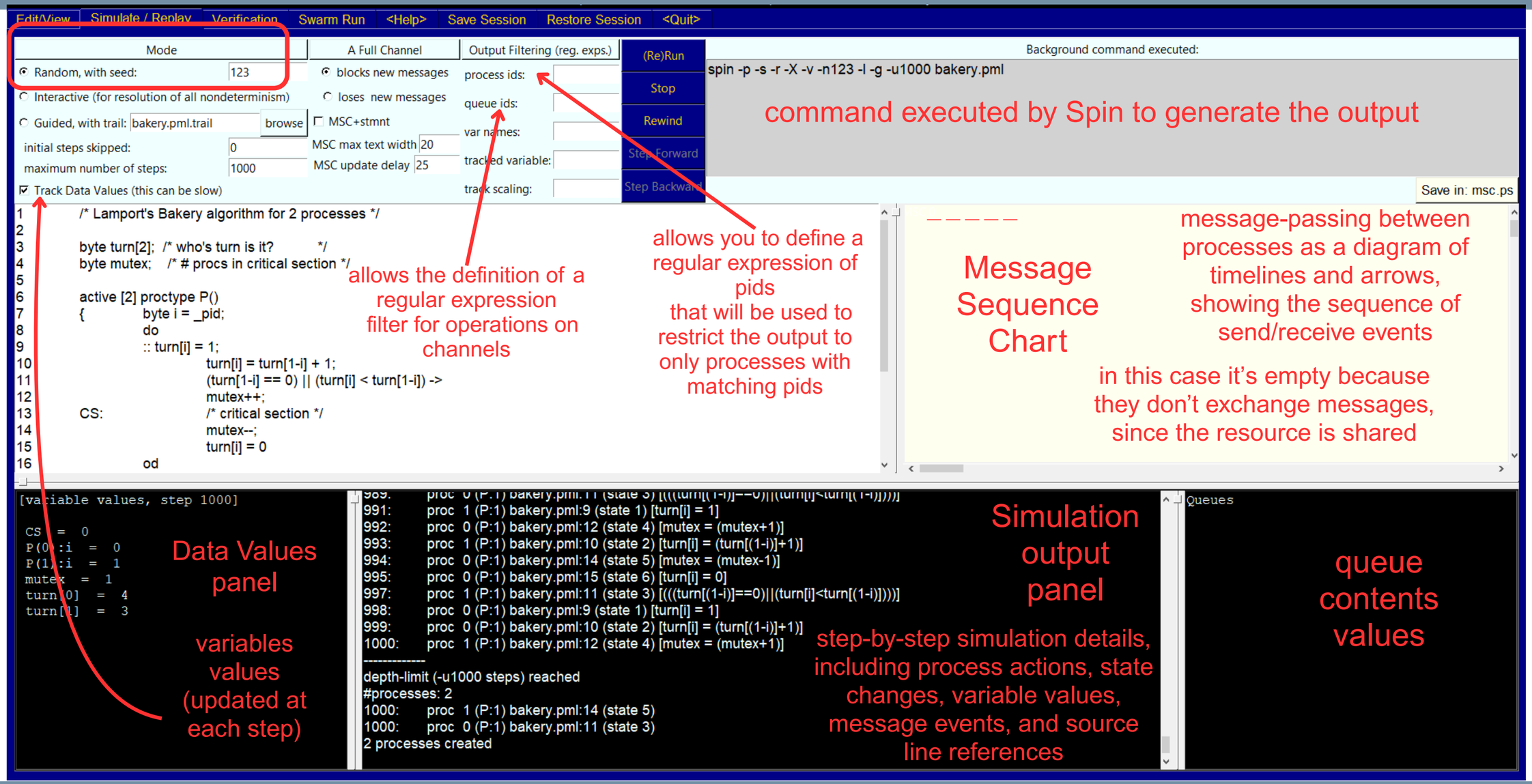

generates automata from the Promela code

# Automata View

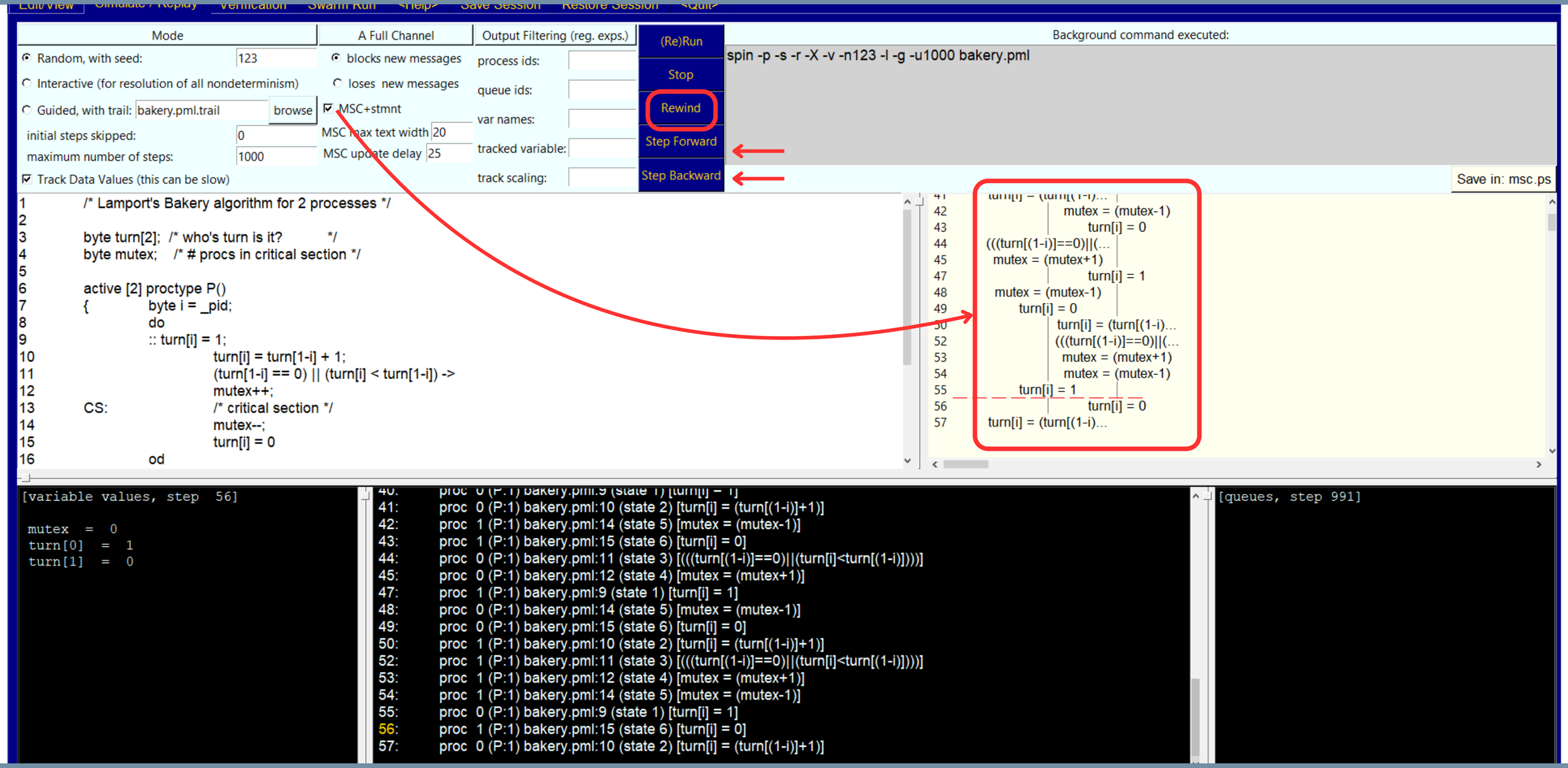


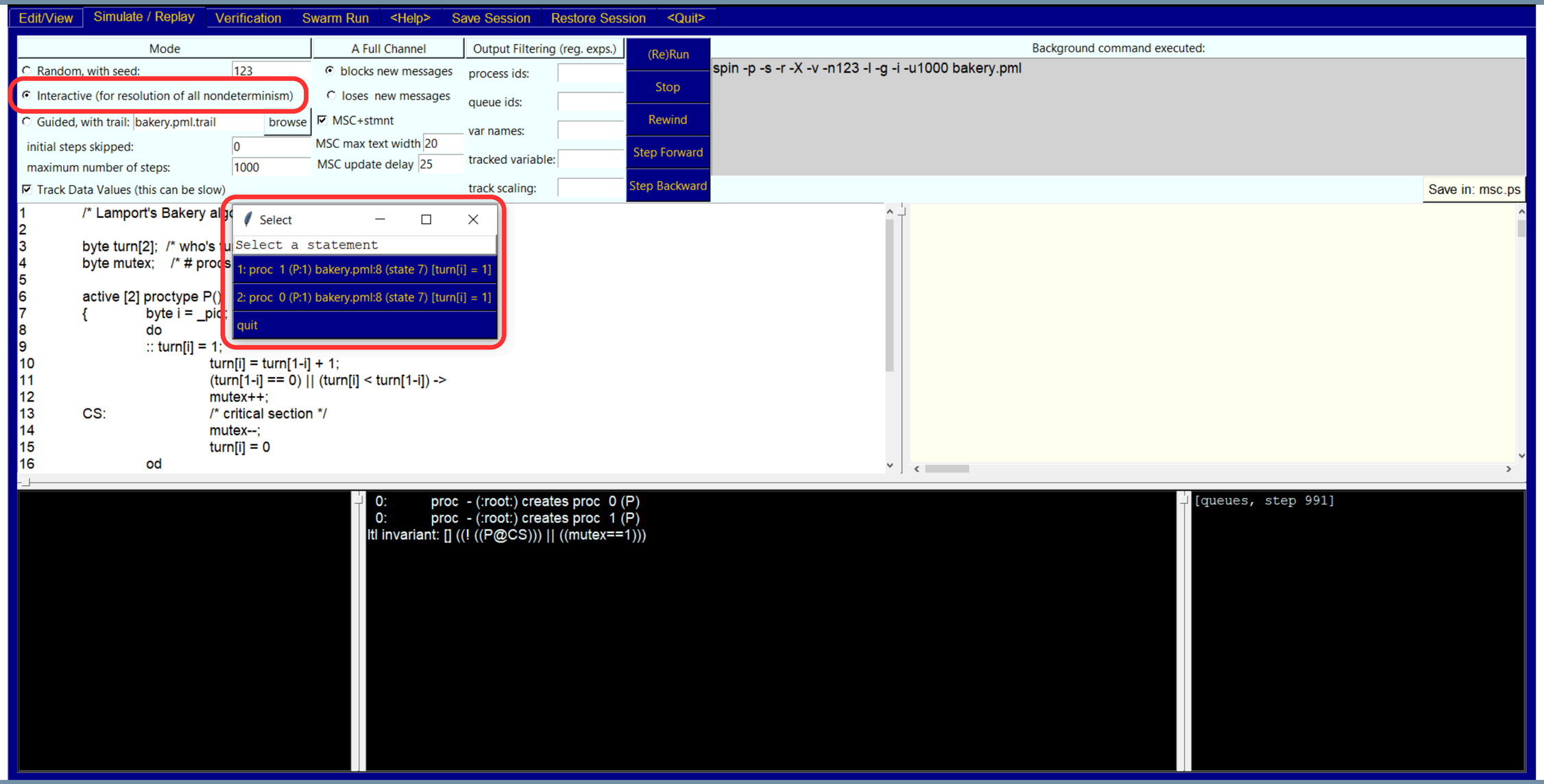
# Simulation - Random



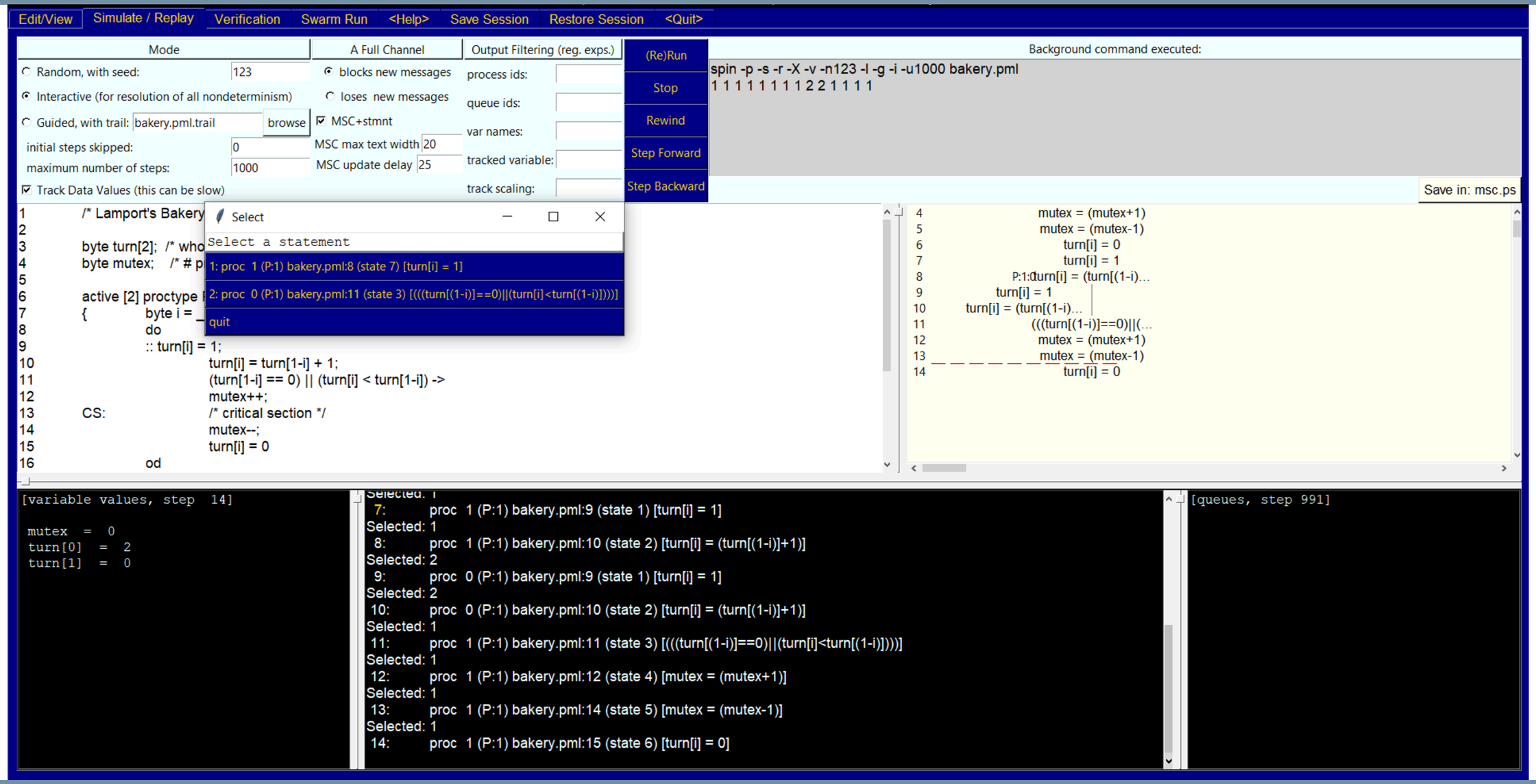
# Simulation - Random



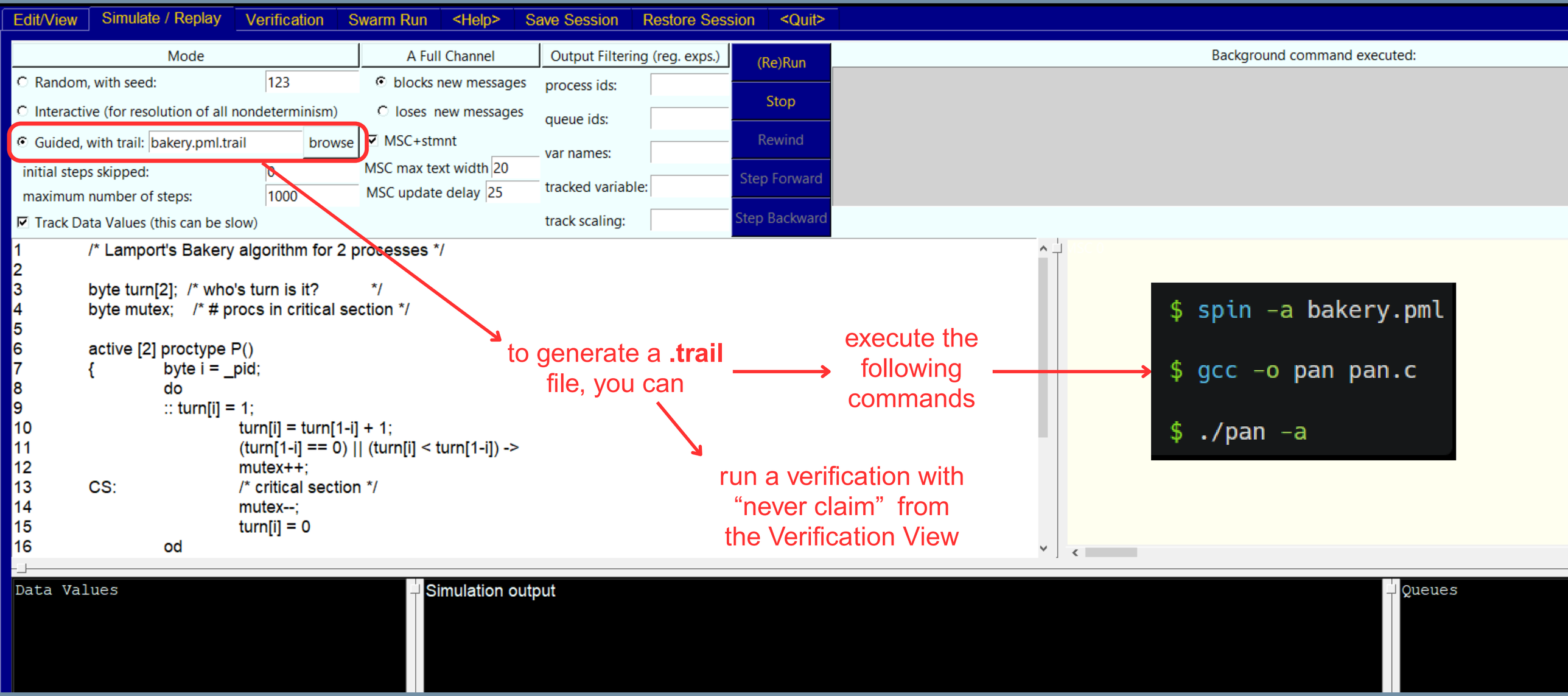
# Simulation - Interactive



# Simulation - Interactive

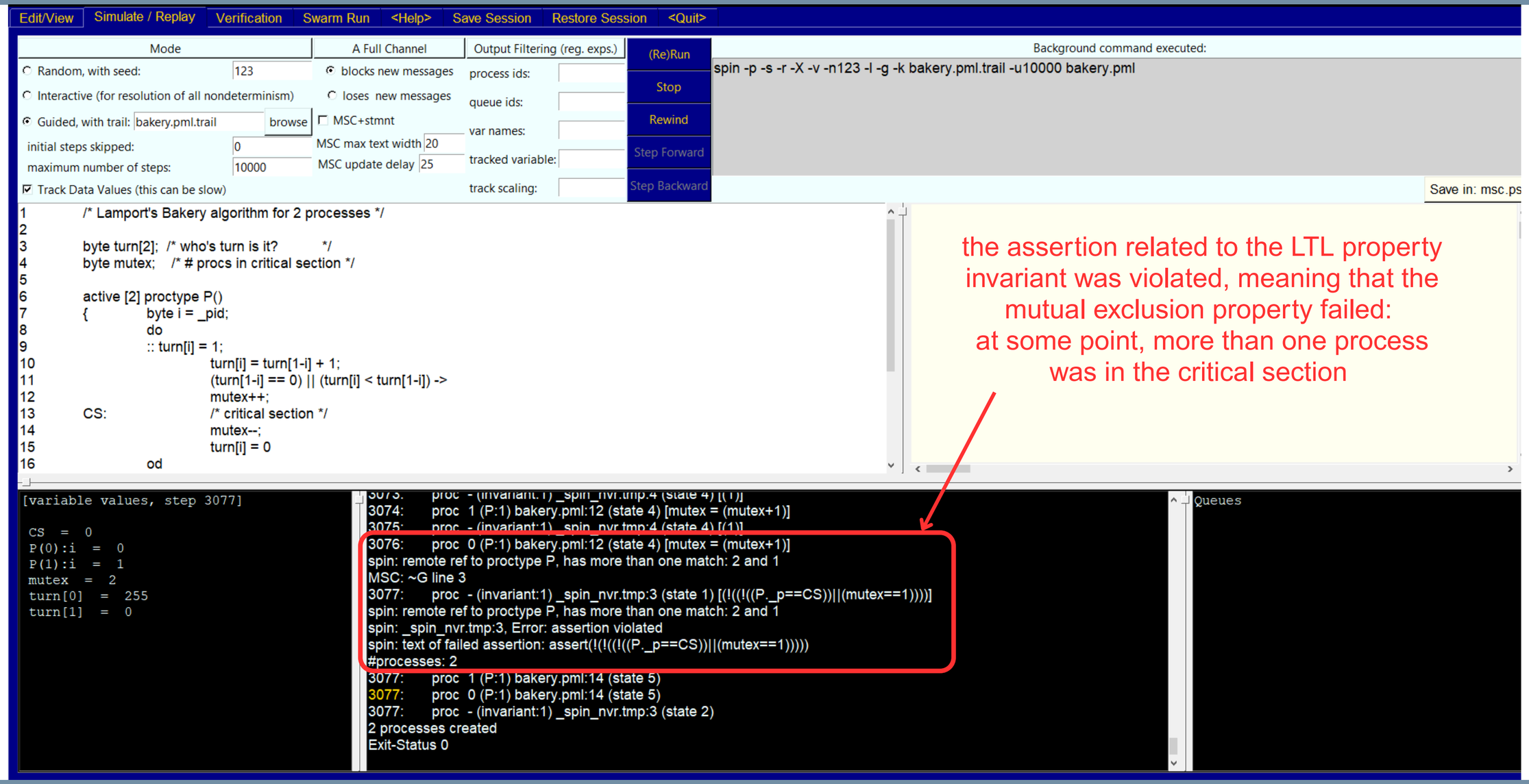


# Simulation - Guided

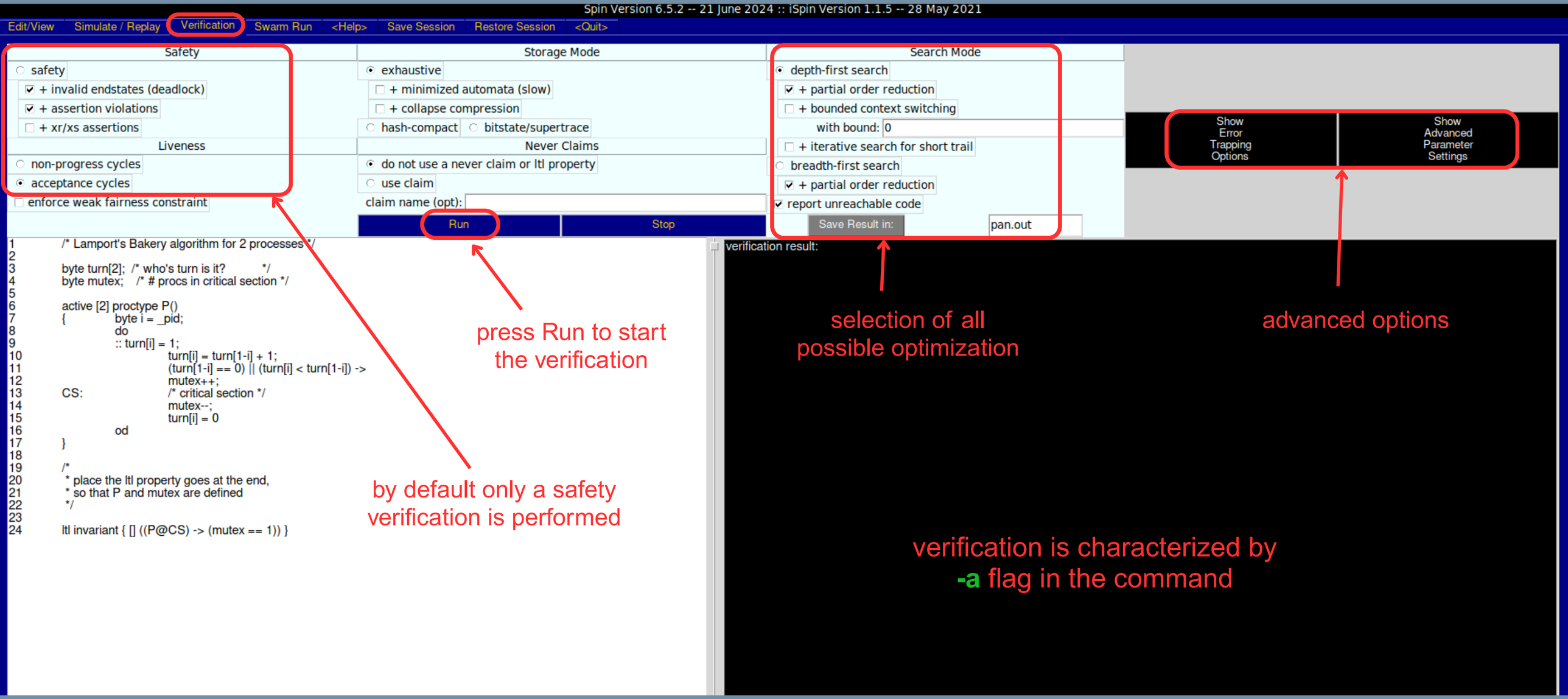



Demo

# Simulation - Guided

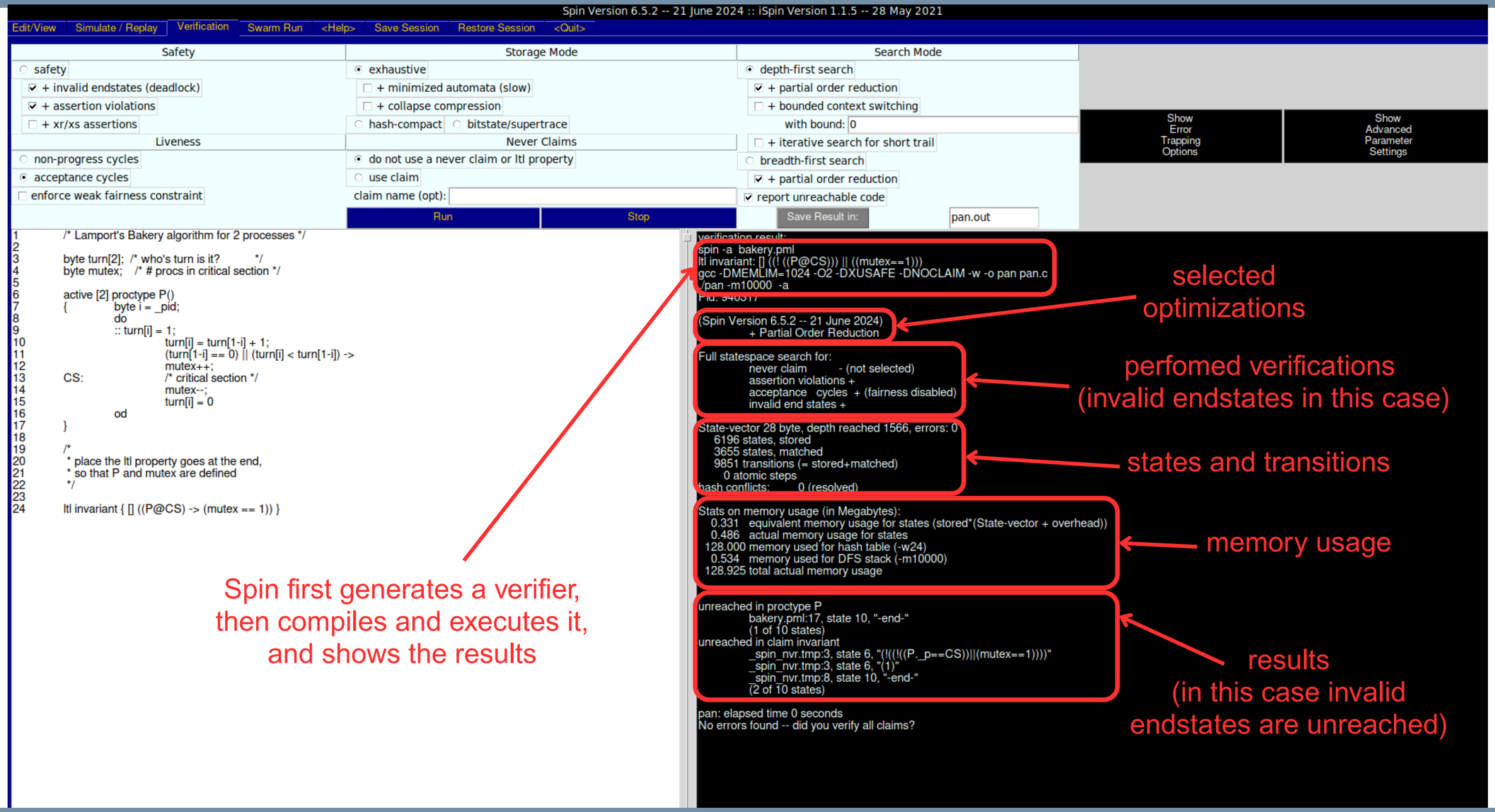


# Verification

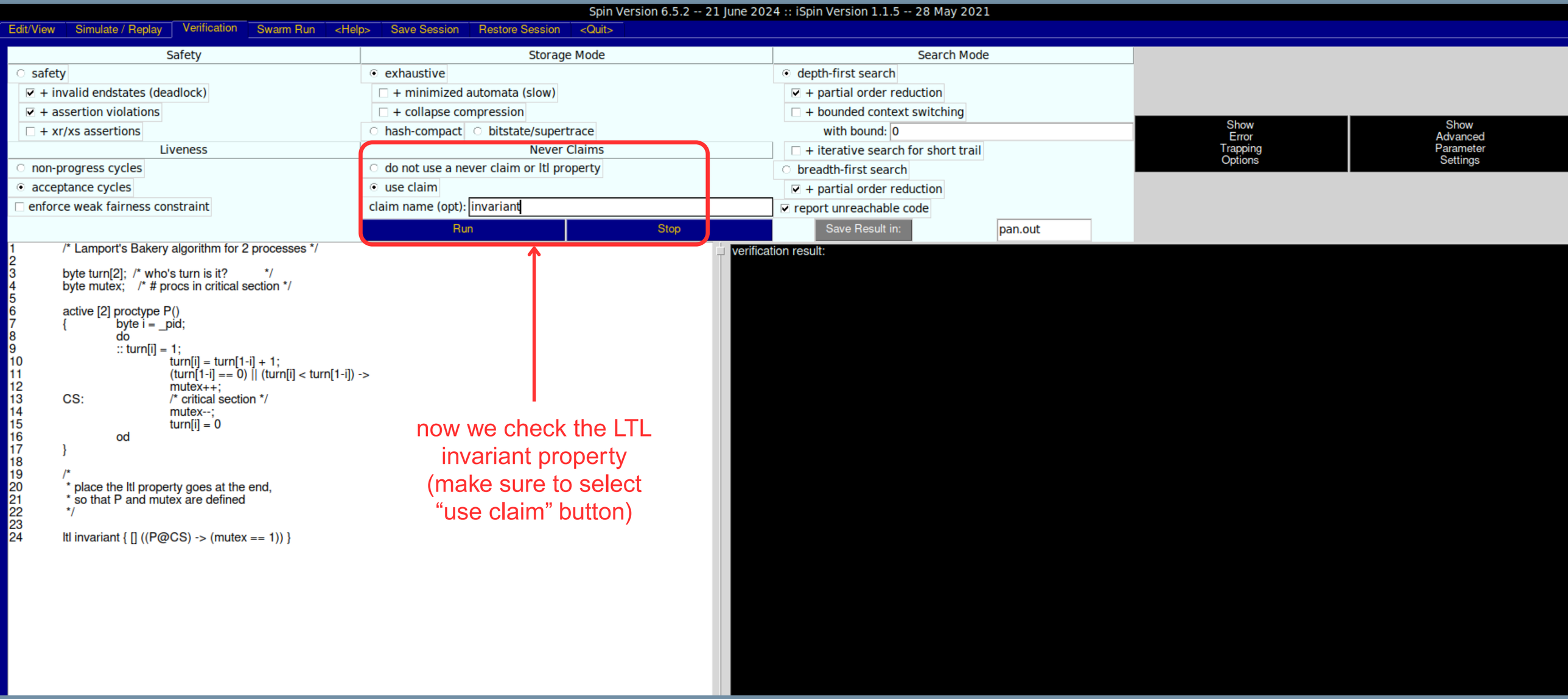



Demo

# Verification

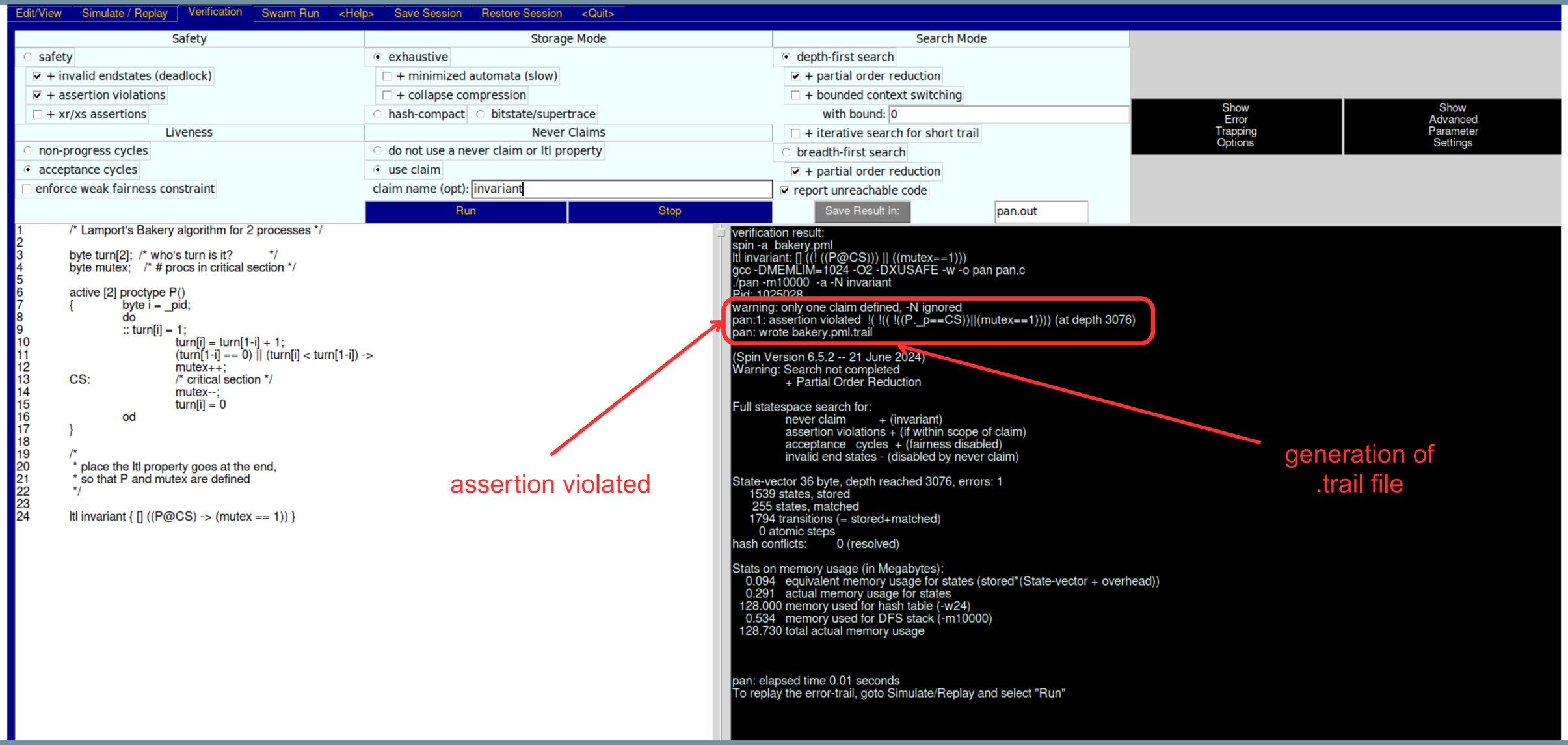


# Verification

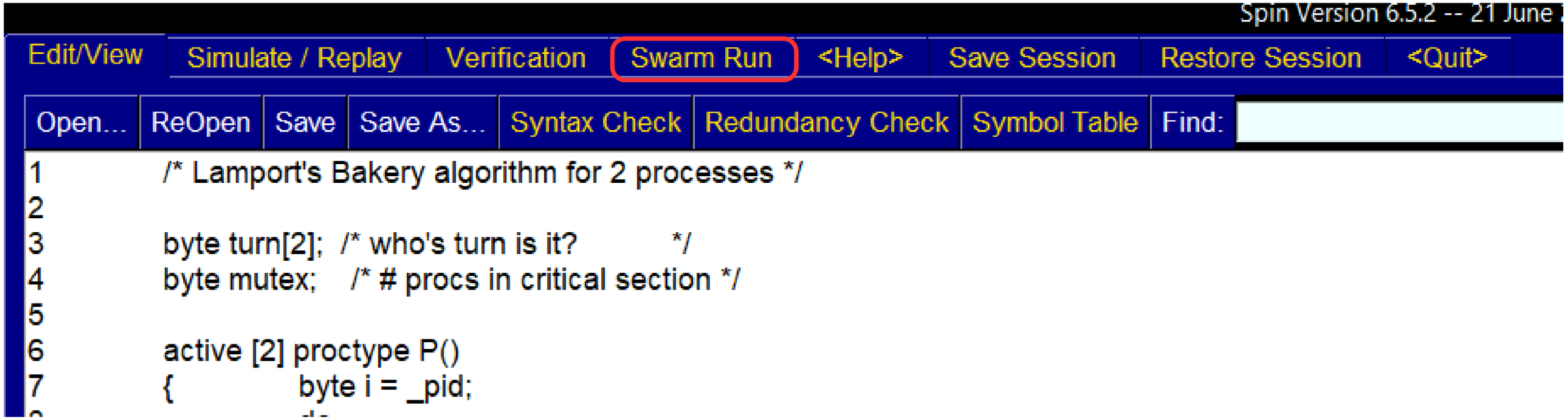



now we check the LTL invariant property (make sure to select “use claim” button)

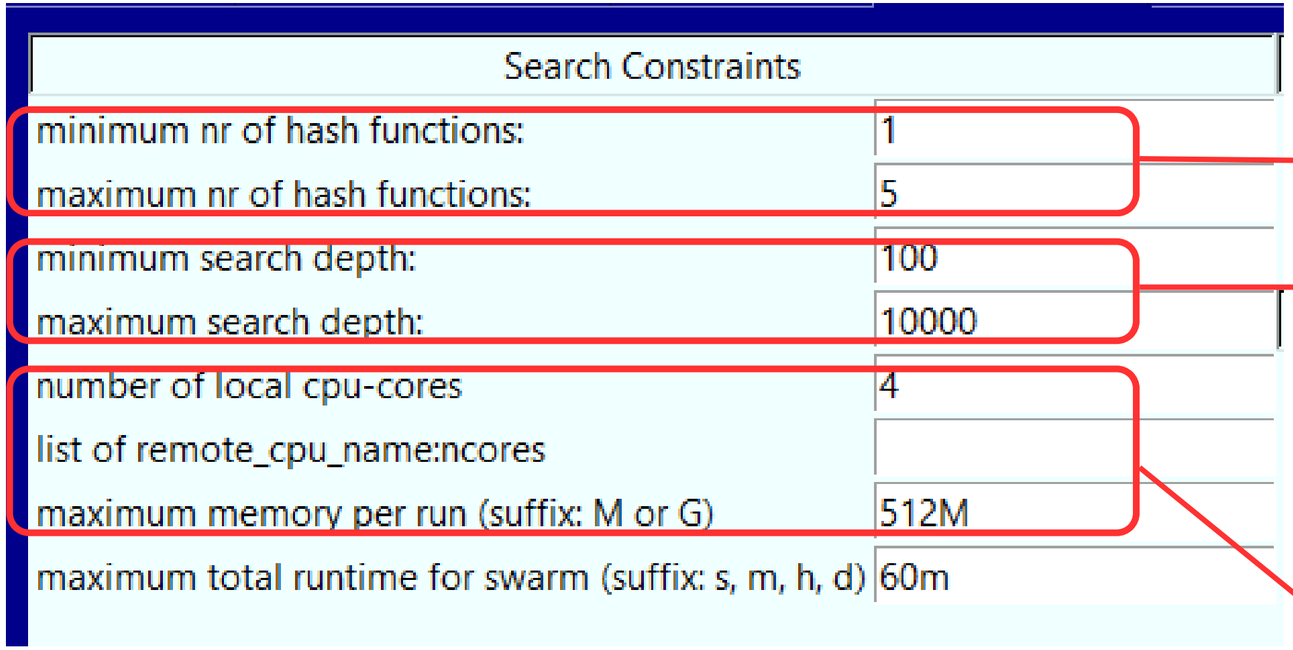
# Verification



# Swarm run


Spin Version 6.5.2 -- 21 June
Edit/View
Simulate / Replay
Verification
Swarm Run
<Help>
Save Session
Restore Session
<Quit>
Open...
ReOpen
Save
Save As...
Syntax Check
Redundancy Check
Symbol Table
Find:
1 /* Lamport's Bakery algorithm for 2 processes */
2
3 byte turn[2]; /* who's turn is it? */
4 byte mutex; /* # procs in critical section */
5
6 active [2] proctype P()
7 { byte i = _pid;

# Swarm run




define the range of number of hash functions to use for the search

define the range of depth of the exploration in the state space tree. setting a minimum force swarm to only start verification after reaching a certain depth, This avoids wasting time on trivial shallow states. Setting a maximum, limit the exploration avoiding infinite paths.

We can define how many processing cores are available and how much memory can be used. Don't select more cores than you actually have on your system and don't select more memory than you actually have

# Swarm run

| Estimates (Fine Tuning) | |
|---|---|
| hash-factor | 1.5 |
| state-vector size in bytes | 512 |
| exploration speed in states/sec | 250000 |
| **Model Generation** | |
| extra spin args | |
| extra pan args | -c1 -x -n |

During Swarm Verification, not every state is necessarily explored (because it would take too long).
SPIN uses a hash-based pruning to randomly skip some states.
hash-factor controls how aggressive that pruning is.
Small hash-factor → more skipping (faster, but may miss bugs).
Large hash-factor → less skipping (slower, but safer).

Each explored state needs memory to store its data.
SPIN usually estimates the size automatically but if we have an idea of the size we can set it to help SPIN avoid running out of memory or underestimating the RAM needed.

SPIN needs to guess how fast it can explore (how many state in a second) to plan the number of random runs.
with this info SPIN can better predict how many randomized search attempts fit inside your time/memory budget.

-c1 → cutoff after 1 error, stop the execution after an error is found (otherwise the search keeps running)
-x → prints useful informations during the search like which states are being explored, good for debugging
-n → suppresses some progress statistics prints (like exploration time) during the run to keep output clean.

# Swarm run

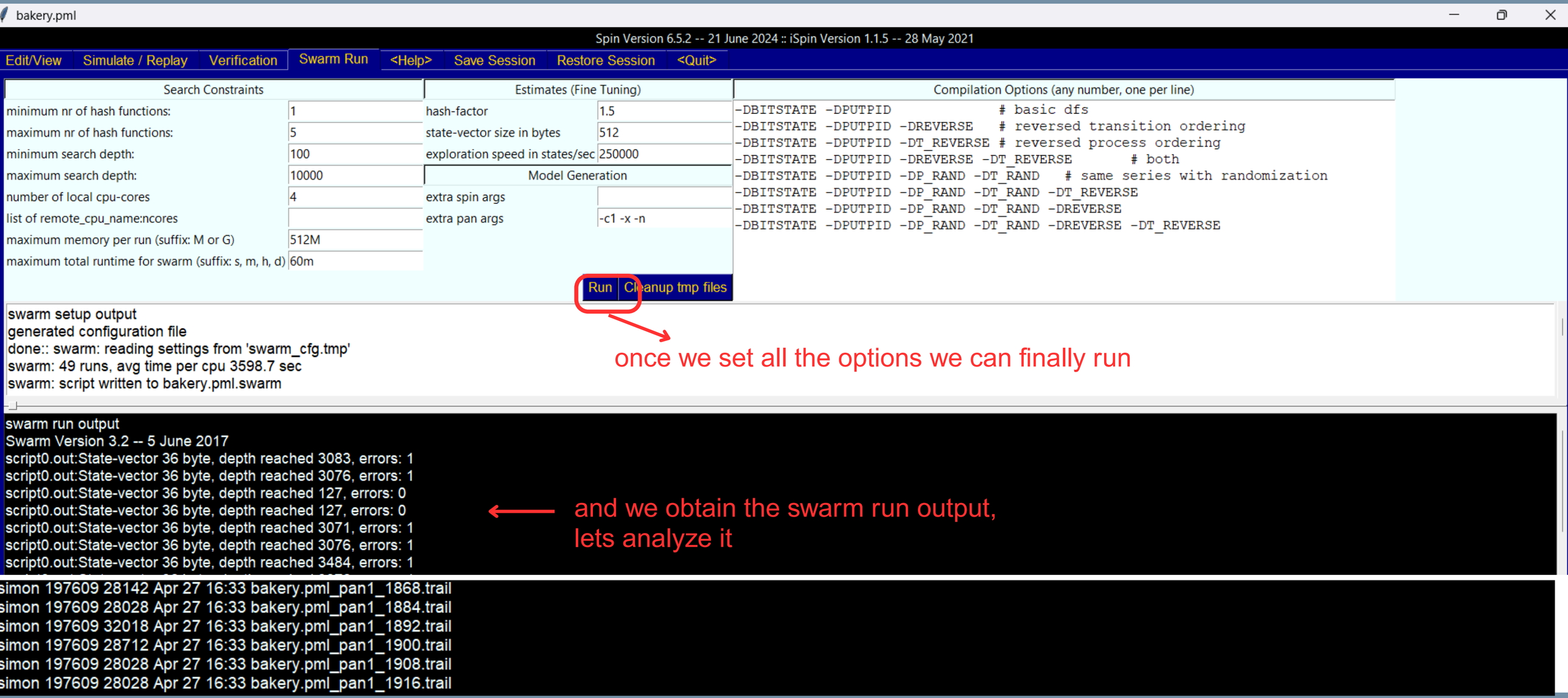
Compilation Options (any number, one per line)

```
-DBITSTATE -DPUTPID                 # basic dfs
-DBITSTATE -DPUTPID -DREVERSE       # reversed transition ordering
-DBITSTATE -DPUTPID -DT_REVERSE # reversed process ordering
-DBITSTATE -DPUTPID -DREVERSE -DT_REVERSE          # both
-DBITSTATE -DPUTPID -DP_RAND -DT_RAND    # same series with randomization
-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DT_REVERSE
-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DREVERSE
-DBITSTATE -DPUTPID -DP_RAND -DT_RAND -DREVERSE -DT_REVERSE
```

This section show how Spin will compile the verifier to create lots of different types of verification runs. This method is especially useful for very large verification models that cannot be exhaustively verified, but where we still want to gain maximum confidence that we can find the errors.

Each line define a slightly different exploration strategy for that particular run.
-DBITSTATE → use bitstate hashing
-DPUTPID → add process IDs into the state description (useful for debugging)
these two are the “basic kit” so are always used and on top we can add other options like:
-DREVERSE → reverse the transition ordering inside each process
-DT_REVERSE → reverse the process execution ordering
-DP_RAND → randomize process execution order
-DT_RAND → randomize transitions inside processes (different transition choices each time).

# Swarm run




once we set all the options we can finally run

and we obtain the swarm run output, lets analyze it

# Swarm run

Size of memory used to represent one system state

```
script0.out:State-vector 36 byte, depth reached 3083, errors: 1
```

Spin divides the total randomized searches across multiple scripts, we have a scriptX for each of the cores (in your case we set 4). Inside each script, multiple randomized searches happen and we will have an output for each one.

When the search finishes, reports depth reached and number of errors. (in your case with the -c1 flag the number of errors will be always 0 or 1)

```
-rw-r--r-- 1 simon 197609 28142 Apr 27 16:33 bakery.pml_pan1_1868.trail
```

When an error occur SPIN creates a trial file wirh the sequence of step that led to the error